

◆ Types of Joins in SQL

1. INNER JOIN

- Returns only the **matching rows** from both tables.

2. LEFT JOIN (LEFT OUTER JOIN)

- Returns **all rows from the left table** + matching rows from right.
- If no match → NULL from right side.

3. RIGHT JOIN (RIGHT OUTER JOIN)

- Opposite of LEFT JOIN.
- Returns all rows from right table + matching rows from left

4. FULL OUTER JOIN

- Returns **all rows from both tables**.
- If no match, fills with NULLs.

⚠ Note: Some databases like MySQL don't support FULL OUTER JOIN directly.

5. CROSS JOIN

- Returns the **Cartesian product** (all possible combinations).

6. SELF JOIN

- A table joins with **itself** (commonly used in hierarchical data).

⚡ Bonus Joins

- **NATURAL JOIN** → Automatically joins tables by columns with the same name. Not widely used because it can create confusion.
 - **EQUI JOIN** → A join using = operator (basically a type of INNER JOIN).
 - **NON-EQUI JOIN** → Joins using <, >, BETWEEN instead of =.
-

✓ **Quick Recap (Remember like this):**

- **INNER JOIN** → Only matches.
- **LEFT JOIN** → All left + matches.
- **RIGHT JOIN** → All right + matches.
- **FULL JOIN** → All rows from both.
- **CROSS JOIN** → All combinations.
- **SELF JOIN** → Table with itself.

◆ **Types of Joins in SQL**

☞ Broadly, we can classify them as:

1. INNER JOIN

- Returns only rows where there's a match in both tables.

2. OUTER JOINS

(They include unmatched rows too)

- **LEFT JOIN (LEFT OUTER JOIN)** → all rows from left + matching from right.
- **RIGHT JOIN (RIGHT OUTER JOIN)** → all rows from right + matching from left.
- **FULL OUTER JOIN** → all rows from both tables (matched + unmatched)

3. CROSS JOIN

- Returns Cartesian product (all combinations).

4. SELF JOIN

- Table joins with itself (hierarchy/recursive data)

5. Special Joins (less common but asked sometimes)

- **NATURAL JOIN** → auto-joins based on same column names (not recommended).
- **EQUI JOIN** → a join using = (actually just a type of INNER JOIN).
- **NON-EQUI JOIN** → join using <, >, BETWEEN etc.

✓ So, in total you can say:

- **6 Core Joins** → INNER, LEFT, RIGHT, FULL, CROSS, SELF.
- **3 Bonus Joins** (theory) → NATURAL, EQUI, NON-EQUI.

```

-- Create Departments table
CREATE TABLE Departments (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50),
    Location VARCHAR(50)
);

-- Create Employees table
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DeptID INT NULL,
    Salary DECIMAL(10,2), --decimal datatype - 10 (total no. of values),
                           2(no. of values after decimal) -- if 10 - 8+2.
                           eg: 17933322.76
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);

-- Insert into Departments
INSERT INTO Departments (DeptID, DeptName, Location) VALUES
(101, 'HR', 'Chennai'),
(102, 'IT', 'Bangalore'),
(104, 'Finance', 'Hyderabad'); -- Notice: 103 is missing intentionally

-- Insert into Employees
INSERT INTO Employees (EmpID, EmpName, DeptID, Salary) VALUES
(1, 'Alice', 101, 50000.00), -- Matches HR
(2, 'Bob', 102, 60000.00), -- Matches IT
(3, 'Charlie', NULL, 55000.00), -- No Dept assigned (instead of 103 for FK safety)
(4, 'David', NULL, 45000.00), -- No Dept assigned
(5, 'Emma', 102, 70000.00); -- Matches IT

select * from Departments;

select * from Employees;

```

```

-- Create Departments table
CREATE TABLE Departments (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50),
    Location VARCHAR(50)
);

-- Create Employees table
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DeptID INT NULL,
    Salary DECIMAL(10,2), --decimal datatype - 10 (total no. of values)
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);

```

```

-- Insert into Departments
INSERT INTO Departments (DeptID, DeptName, Location) VALUES
(101, 'HR', 'Chennai'),
(102, 'IT', 'Bangalore'),
(104, 'Finance', 'Hyderabad'); -- Notice: 103 is missing intentionally

-- Insert into Employees
INSERT INTO Employees (EmpID, EmpName, DeptID, Salary) VALUES
(1, 'Alice', 101, 50000.00), -- Matches HR
(2, 'Bob', 102, 60000.00), -- Matches IT
(3, 'Charlie', NULL, 55000.00), -- No Dept assigned (instead of 103 for FK safety)
(4, 'David', NULL, 45000.00), -- No Dept assigned
(5, 'Emma', 102, 70000.00); -- Matches IT

select * from Departments;
select * from Employees;

```

83 %

Results		Messages	
	DeptID	DeptName	Location
1	101	HR	Chennai
2	102	IT	Bangalore
3	104	Finance	Hyderabad

	EmpID	EmpName	DeptID	Salary
1	1	Alice	101	50000.00
2	2	Bob	102	60000.00
3	3	Charlie	NULL	55000.00
4	4	David	NULL	45000.00
5	5	Emma	102	70000.00

INNER JOIN

Brings only **matching rows** between two tables based on the join condition.

Rows without a match in either table are excluded.

Basic INNER JOIN Query

```

SELECT e.EmpID, e.EmpName, d.DeptName, d.Location
FROM Employees e
INNER JOIN Departments d
ON e.DeptID = d.DeptID;

```

Joined based on condition – with common column values

```

SELECT e.EmpID, e.EmpName, d.DeptName, d.Location
FROM Employees e
INNER JOIN Departments d
ON e.DeptID = d.DeptID;

```

33 %

Results Messages

	EmpID	EmpName	DeptName	Location
1	1	Alice	HR	Chennai
2	2	Bob	IT	Bangalore
3	5	Emma	IT	Bangalore

- Alice, Bob, Emma have matching departments → included ✓
- Charlie, David (DeptID = NULL) → excluded ✗
- Finance (104) has no employee → excluded ✗

◆ Practice Queries for INNER JOIN

1 List all employees with their department name

```

select e.EmpName, d.DeptName
from Employees E
inner join Departments D
on e.DeptId=d.DeptId;

```

```

-- Practice Queries for INNER JOIN

-- 1. List all employees with their department name

select e.EmpName, d.DeptName
from Employees E
inner join Departments D
on e.DeptId=d.DeptId;

```

91 %

Results Messages

	EmpName	DeptName
1	Alice	HR
2	Bob	IT
3	Emma	IT

2. Get employee names and their department locations

```
select e.EmpName, d.location
from employees e
inner join departments d
on e.deptid=d.deptid
```

```
-- 2. Get employee names and their department locations
```

```
select e.EmpName, d.location
from employees e
inner join departments d
on e.deptid=d.deptid
```

91 %

Results Messages

	EmpName	location
1	Alice	Chennai
2	Bob	Bangalore
3	Emma	Bangalore

3. Find all employees in the IT department

```
select e.EmpName, d.DeptName
from Employees E
inner join Departments D
on e.DeptId=d.DeptId
where DeptName='IT';
```

```
-- 3. Find all employees in the IT department
```

```
select e.EmpName, d.DeptName
from Employees E
inner join Departments D
on e.DeptId=d.DeptId
where DeptName='IT';
```

91 %

Results Messages

	EmpName	DeptName
1	Bob	IT
2	Emma	IT

4 Count how many employees are in each department

```
select DeptName, count(EmpId) as total from departments d
inner join employees e
on d.deptid=e.deptid
group by DeptName;
```

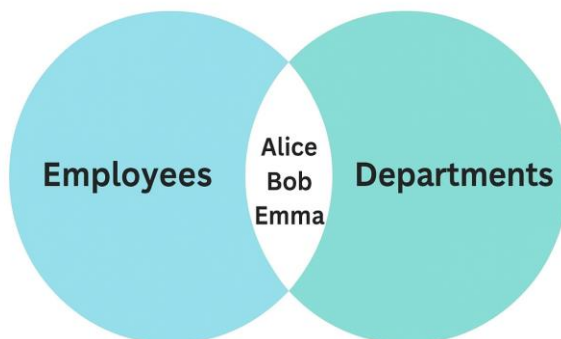
```
-- 4. Count how many employees are in each department
select DeptName, count(EmpId) as total from departments d
inner join employees e
on d.deptid=e.deptid
group by DeptName;
```

91 %

Results Messages

	DeptName	total
1	HR	1
2	IT	2

INNER JOIN



when to use **INNER JOIN** vs **LEFT/RIGHT JOIN** vs **FULL JOIN** is the real skill.

Let me break it down with a **decision-making guide** 📌

◆ INNER JOIN

- Use when you need **only matching records** between both tables.
 - Excludes rows that don't have a match.
 - Example: "Show all employees who belong to a valid department."
☞ Charlie (NULL DeptID), David (NULL DeptID), and Finance (no employees) won't appear.
-

◆ LEFT JOIN

- Use when you want **all records from the left table**, plus matching ones from the right.
 - If no match exists, NULLs are returned for the right side.
 - Example: "List all employees and their departments (even if some employees don't belong to any dept)."
☞ Charlie & David will appear with DeptName = NULL.
-

◆ RIGHT JOIN

- Opposite of LEFT JOIN: **all records from the right table**, plus matching ones from the left.
 - Example: "List all departments and the employees (even if no employee belongs to a department)."
☞ Finance department will appear with Employee = NULL.
-

◆ FULL OUTER JOIN

- Use when you want **all records from both tables**, matched wherever possible.
 - Example: "Show all employees and all departments, even if they don't match."
☞ You'll see Charlie, David, and Finance all included.
-

⚡ **Quick Visual:**

- **INNER JOIN** → Intersection only.
 - **LEFT JOIN** → Left circle + intersection.
 - **RIGHT JOIN** → Right circle + intersection.
 - **FULL JOIN** → Entire union of both circles.
-

🔗 How to Decide

Ask yourself:

1. Do I care only about **rows that exist in both tables**?
→ **INNER JOIN**
 2. Do I want **all rows from left table no matter what**?
→ **LEFT JOIN**
 3. Do I want **all rows from right table no matter what**?
→ **RIGHT JOIN**
 4. Do I want **everything from both sides**?
→ **FULL OUTER JOIN**
-

☞ In your **count query**, since you only cared about departments **with employees**, `INNER JOIN` is correct.

If the requirement was: *“Show all departments, even those with 0 employees”* → you’d use `LEFT JOIN`.

bt for all the 4 practice questions which we practiced can be done normally also without join operation ryt ? or it wont work on 2 different tables ah ?

◆ If everything was in one single table

Yes ✓ — all those 4 questions could be done **without JOIN** (just with `WHERE`, `GROUP BY`, etc.), because all data would already be in one place.

☞ Example: If `DeptName` and `Location` were stored inside the `Employees` table itself, you could simply query them directly.

◆ But in real-world databases

We usually **normalize** data → meaning we split information into multiple related tables (to avoid duplication, make maintenance easier).

- Employees info → `Employees` table
- Departments info → `Departments` table

If you want an employee's department name or location → you **must combine** the two tables.

And the only way to combine data from different tables in SQL is through **JOINS** (or subqueries, but internally that still works like a join).

⚡ Why JOIN is needed here

Let's check our 4 practice queries:

1 List all employees with their department name

- DeptName lives in `Departments`, EmpName lives in `Employees`.
- Without JOIN, you can't connect them.

2 Get employee names and their department locations

- Again, Location is in `Departments`, EmpName in `Employees`.
- Needs JOIN.

3 Find all employees in the IT department

- DeptName = 'IT' → this info is only in `Departments` table.
- Must JOIN to `Employees`.

4 Count how many employees are in each department

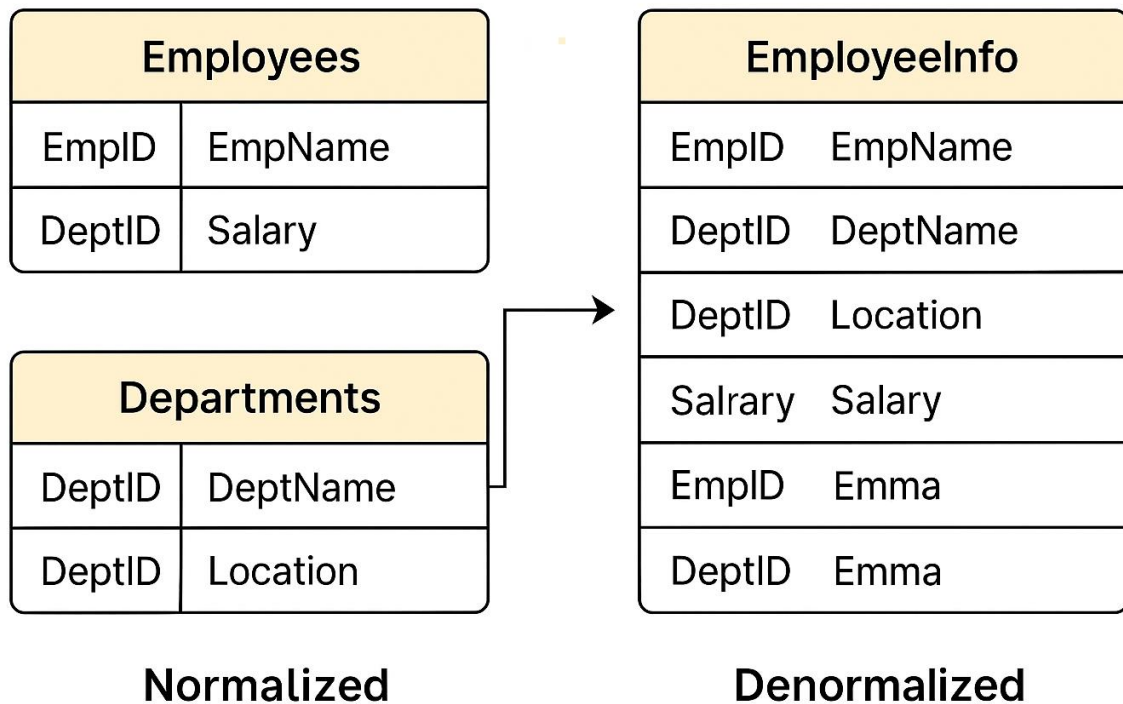
- Employees are in one table, Departments are in another.
- Must JOIN to group employees by dept.

If everything was denormalized (all info in one table), JOIN wouldn't be required.

But since info is **split across tables**, JOIN is **the correct and only scalable way**.

💡 Rule of thumb:

If your query needs **columns from more than one table** → you need a JOIN (or a subquery that mimics one).



But here's the problem with denormalized design:

- DeptName and Location are **repeated** for every employee in IT.
- If the company changes IT's location (say from Bangalore → Pune), you'd have to update it in multiple rows instead of just once.
- That's why **normalization** is preferred → store departments separately, and link using DeptID.

☞ So:

- **One big table** → **simpler queries (no JOINS)** but poor data integrity.
- **Normalized tables** → **requires JOINS** but cleaner, scalable, and easier to maintain.

What is an OUTER JOIN?

- An **OUTER JOIN** returns not only the **matching rows** between two tables (like INNER JOIN) but also the **non-matching rows** from one or both tables.
- Wherever there is no match, the missing side will be filled with **NULL**.

☞ So, **OUTER JOIN = MATCHES + UNMATCHES (with NULLs)**.

◆ Types of OUTER JOINS

Outer joins are divided into **3 categories**, depending on *which side's unmatched rows you want to keep*:

1. **LEFT OUTER JOIN (LEFT JOIN)**
 - Keeps all rows from the **left table**.
 - Matches from the right side are shown if they exist, else NULL.
 2. **RIGHT OUTER JOIN (RIGHT JOIN)**
 - Keeps all rows from the **right table**.
 - Matches from the left side are shown if they exist, else NULL.
 3. **FULL OUTER JOIN (FULL JOIN)**
 - Keeps **all rows from both tables**.
 - Shows matches where available, else NULLs for the missing side.
-

◆ Why INNER JOIN doesn't have subcategories?

- INNER JOIN is **only the intersection** (the common part) between two tables.
- It does **not care about unmatched rows** at all.
- Since there's no concept of "keep left" or "keep right" (because all unmatched rows are dropped anyway), there's only **one type of INNER JOIN**.

☞ OUTER JOIN needs 3 categories because we can decide:

- keep left unmatched rows,
- keep right unmatched rows,
- or keep both.

But INNER JOIN = intersection → only **one way to do it**.

🔗 Quick Visual:

- **INNER JOIN** → intersection only.
 - **LEFT OUTER JOIN** → left circle + intersection.
 - **RIGHT OUTER JOIN** → right circle + intersection.
 - **FULL OUTER JOIN** → entire union (both circles).
-

↗ So:

- **INNER JOIN** is "strict": only matches.
 - **OUTER JOIN** is "inclusive": we decide which unmatched rows we also want.
-

🌟 LEFT JOIN (also called **LEFT OUTER JOIN**)

◆ Definition

- • ✓ All rows from the **left table** (even if no match in the right table).
- • ✓ Matching rows from the **right table**.
- • ✗ If no match, the right table's columns will show **NULL**

◆ Syntax

```
SELECT columns
FROM LeftTable L
LEFT JOIN RightTable R
ON L.CommonColumn = R.CommonColumn;
```

```
-- Query 1: List all employees with their department names
```

```
select EmpName, DeptName
from Employees E
left join Departments D
on e.deptId=d.deptId;
```

```
-- Query 1: List all employees with their department names
```

```
select EmpName, DeptName
from Employees E
left join Departments D
on e.deptId=d.deptId;
```

91 %

	EmpName	DeptName
1	Alice	HR
2	Bob	IT
3	Charlie	NULL
4	David	NULL
5	Emma	IT

-- Q2. List all departments with the employees working in them (if no employees, show NULL).

```
select deptname, empname
from departments d
left join employees e
on d.deptId=e.deptId;
```

```
-- Q2. List all departments with the employees working in them (if no employees, show NULL).
```

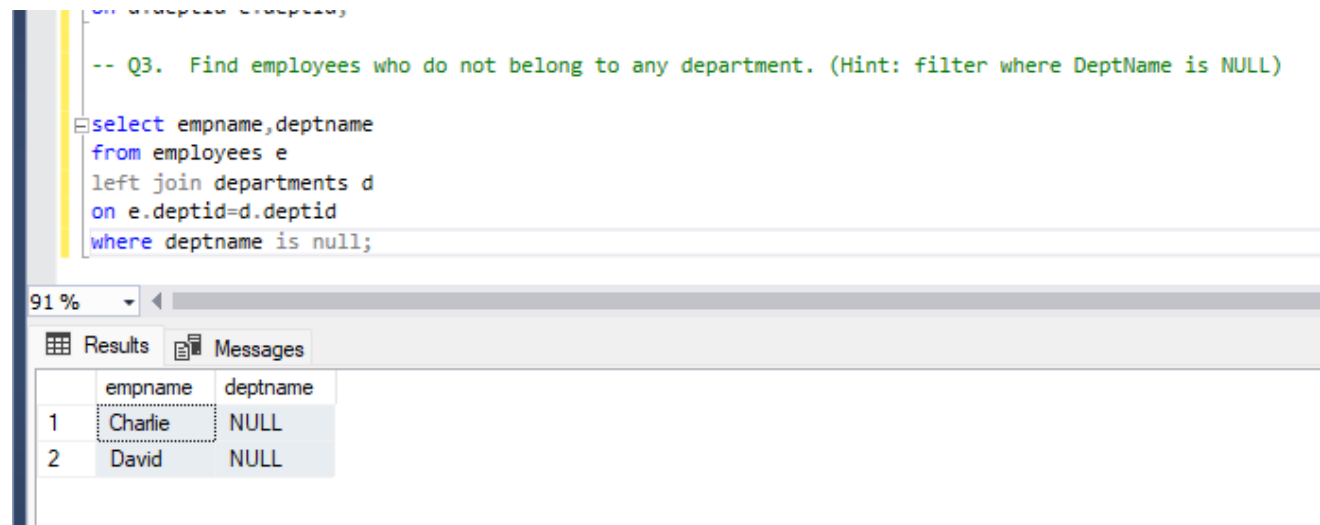
```
select deptname, empname
from departments d
left join employees e
on d.deptId=e.deptId;
```

91 %

	deptname	empname
1	HR	Alice
2	IT	Bob
3	IT	Emma
4	Finance	NULL

-- Q3. Find employees who do not belong to any department. (Hint: filter where DeptName is NULL)

```
select empname,deptname
from employees e
left join departments d
on e.deptid=d.deptid
where deptname is null;
```



The screenshot shows a SQL query editor with the following text:

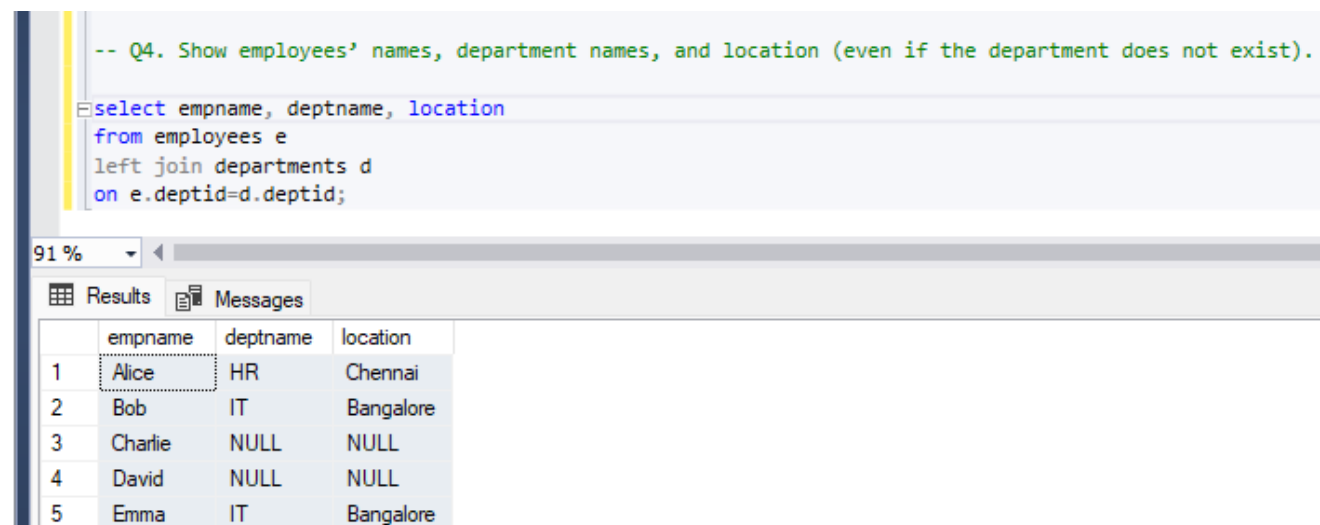
```
-- Q3. Find employees who do not belong to any department. (Hint: filter where DeptName is NULL)
select empname,deptname
from employees e
left join departments d
on e.deptid=d.deptid
where deptname is null;
```

Below the editor, the 'Results' tab is active, displaying a table with 2 rows and 2 columns: empname and deptname.

	empname	deptname
1	Charlie	NULL
2	David	NULL

-- Q4. Show employees' names, department names, and location (even if the department does not exist).

```
select empname, deptname, location
from employees e
left join departments d
on e.deptid=d.deptid;
```



The screenshot shows a SQL query editor with the following text:

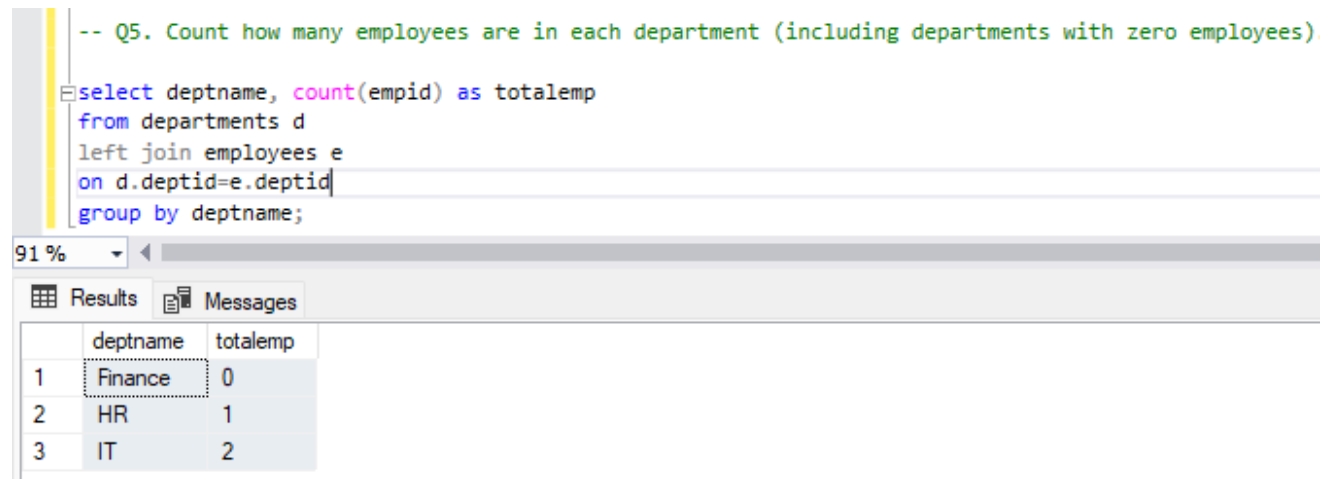
```
-- Q4. Show employees' names, department names, and location (even if the department does not exist).
select empname, deptname, location
from employees e
left join departments d
on e.deptid=d.deptid;
```

Below the editor, the 'Results' tab is active, displaying a table with 5 rows and 4 columns: empname, deptname, and location.

	empname	deptname	location
1	Alice	HR	Chennai
2	Bob	IT	Bangalore
3	Charlie	NULL	NULL
4	David	NULL	NULL
5	Emma	IT	Bangalore

-- Q5. Count how many employees are in each department (including departments with zero employees).

```
select deptname, count(empid) as totalemp
from departments d
left join employees e
on d.deptid=e.deptid
group by deptname;
```



The screenshot shows a SQL IDE with a query editor and a results pane. The query editor contains the following SQL code:

```
-- Q5. Count how many employees are in each department (including departments with zero employees)
select deptname, count(empid) as totalemp
from departments d
left join employees e
on d.deptid=e.deptid
group by deptname;
```

The results pane shows a table with two columns: deptname and totalemp. The table contains three rows of data:

	deptname	totalemp
1	Finance	0
2	HR	1
3	IT	2

What is RIGHT JOIN?

☞ A **RIGHT JOIN** (Right Outer Join) returns:

- ✓ **All rows from the RIGHT table**
- ✓ Matching rows from the LEFT table
- ✗ If no match is found, columns from the LEFT table are returned as **NULL**

In short:

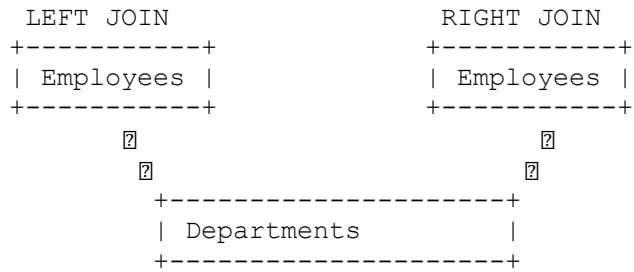
- **Keep everything from RIGHT side**
- Match from LEFT if possible
- Else → show NULLs

It's just the **mirror image** of LEFT JOIN

◆ Syntax

```
SELECT columns
FROM LeftTable l
RIGHT JOIN RightTable r
ON l.CommonColumn = r.CommonColumn;
```

◆ Visual (mirror of LEFT JOIN)



- LEFT JOIN → Keep all from Employees
- RIGHT JOIN → Keep all from Departments

-- Q1. List all departments and employees (even if no employee exists in a department).

```
select deptname, empname
from employees e
right join departments d
on e.deptid=d.deptid;
```

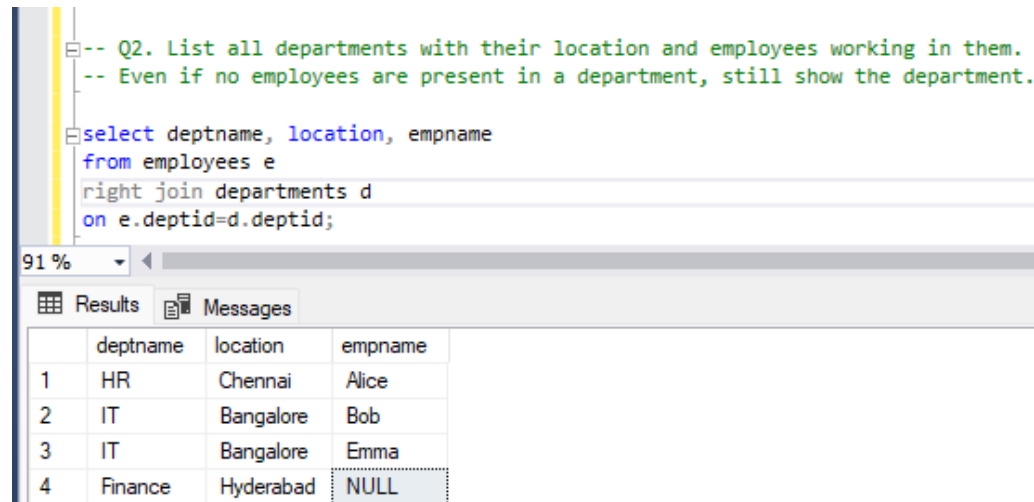
```
-- RIGHT JOIN (Right Outer Join)
-- Q1. List all departments and employees (even if no employee exists in a department).

select deptname, empname
from employees e
right join departments d
on e.deptid=d.deptid;
```

	deptname	empname
1	HR	Alice
2	IT	Bob
3	IT	Emma
4	Finance	NULL

-- Q2. List all departments with their location and employees working in them.
-- Even if no employees are present in a department, still show the department.

```
select deptname, location, empname  
from employees e  
right join departments d  
on e.deptid=d.deptid;
```

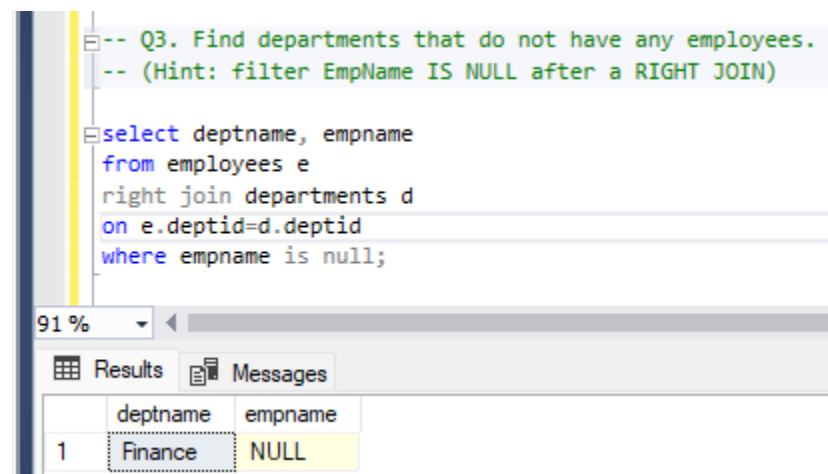


```
-- Q2. List all departments with their location and employees working in them.  
-- Even if no employees are present in a department, still show the department.  
  
select deptname, location, empname  
from employees e  
right join departments d  
on e.deptid=d.deptid;
```

	deptname	location	empname
1	HR	Chennai	Alice
2	IT	Bangalore	Bob
3	IT	Bangalore	Emma
4	Finance	Hyderabad	NULL

-- Q3. Find departments that do not have any employees.
-- (Hint: filter EmpName IS NULL after a RIGHT JOIN)

```
select deptname, empname  
from employees e  
right join departments d  
on e.deptid=d.deptid  
where empname is null;
```



```
-- Q3. Find departments that do not have any employees.  
-- (Hint: filter EmpName IS NULL after a RIGHT JOIN)  
  
select deptname, empname  
from employees e  
right join departments d  
on e.deptid=d.deptid  
where empname is null;
```

	deptname	empname
1	Finance	NULL

-- Q4. Show all departments and employees, along with the department's location (include empty departments).

```
select deptname, empname, location
from employees e
right join departments d
on e.deptid=d.deptid;
```

```
-- Q4. Show all departments and employees, along with the department's location (include empty departments).
select deptname, empname, location
from employees e
right join departments d
on e.deptid=d.deptid;
```

	deptname	empname	location
1	HR	Alice	Chennai
2	IT	Bob	Bangalore
3	IT	Emma	Bangalore
4	Finance	NULL	Hyderabad

-- Q5. Count how many employees are in each department, including departments with zero employees (using RIGHT JOIN this time).

```
select deptname, count(empname) as totemp
from employees e
right join departments d
on e.deptid=d.deptid
group by deptname;
```

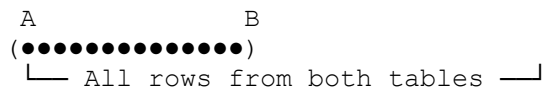
```
-- Q5. Count how many employees are in each department, including departments with zero employees (using RIGHT JOIN this time).
select deptname, count(empname) as totemp
from employees e
right join departments d
on e.deptid=d.deptid
group by deptname;
```

	deptname	totemp
1	Finance	0
2	HR	1
3	IT	2

🌐 What is Full Outer Join?

- A **FULL OUTER JOIN** returns **all rows from both tables**:
 - If there's a match → show matching data
 - If no match → show `NULL` for missing side
 - Think of it as **LEFT JOIN + RIGHT JOIN combined**
-

Venn Diagram (easy to remember)



◆ Syntax:

```
SELECT e.EmpName, d.DeptName
FROM Employees e
FULL OUTER JOIN Departments d
ON e.DeptID = d.DeptID;
```

In SQL:

- `LEFT JOIN` is the same as `LEFT OUTER JOIN`
- `RIGHT JOIN` is the same as `RIGHT OUTER JOIN`
- `FULL JOIN` is the same as `FULL OUTER JOIN`

☞ The keyword **OUTER** is optional.

Most developers just write `LEFT JOIN`, `RIGHT JOIN`, `FULL JOIN` because it's shorter.

So both work fine:

```
-- Same meaning
SELECT * FROM Employees e LEFT JOIN Departments d ON e.DeptID = d.DeptID;

SELECT * FROM Employees e LEFT OUTER JOIN Departments d ON e.DeptID = d.DeptID;
```

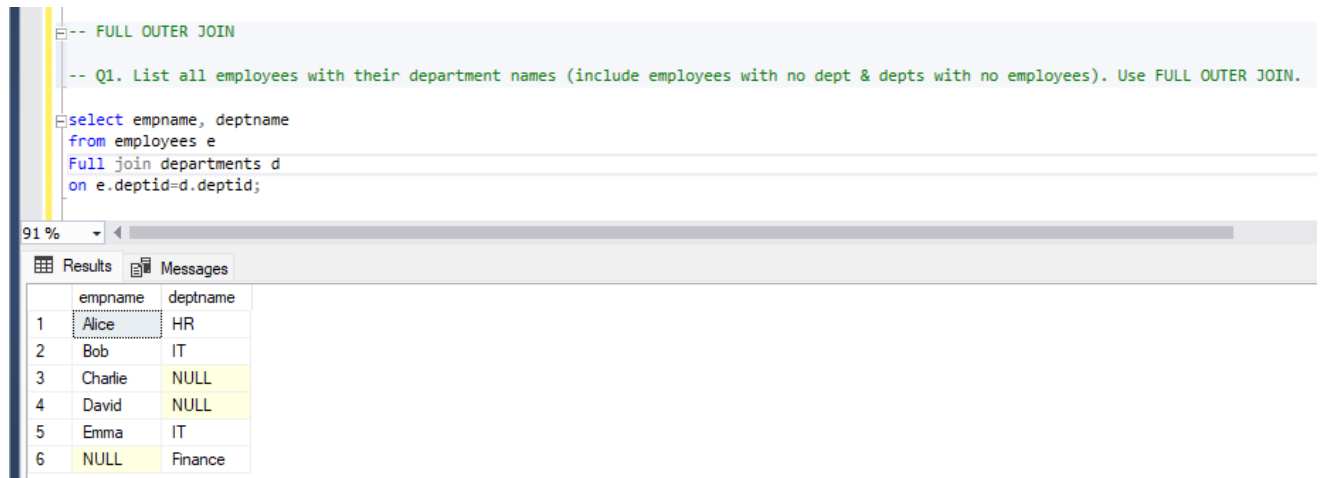
💡 For interview perspective:

- Always mention “**Outer**” when explaining (shows clarity).
- But in coding, you can skip it (common practice).

-- FULL OUTER JOIN

-- Q1. List all employees with their department names (include employees with no dept & depts with no employees). Use FULL OUTER JOIN.

```
select empname, deptname
from employees e
Full join departments d
on e.deptid=d.deptid;
```



The screenshot shows a SQL query editor with the following text:

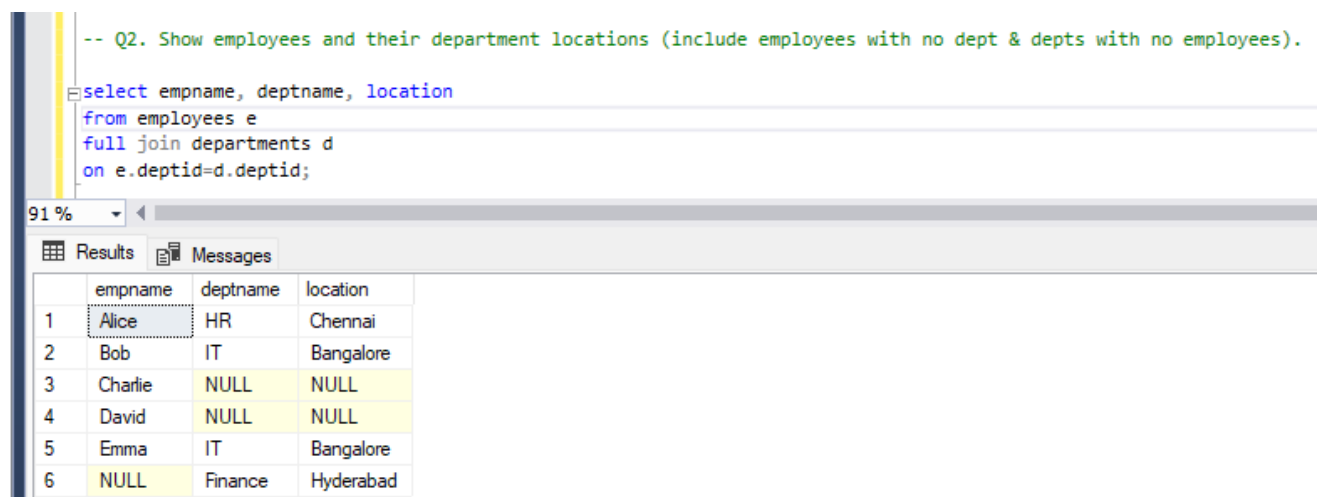
```
-- FULL OUTER JOIN
-- Q1. List all employees with their department names (include employees with no dept & depts with no employees). Use FULL OUTER JOIN.
select empname, deptname
from employees e
Full join departments d
on e.deptid=d.deptid;
```

Below the editor, the 'Results' tab is active, displaying a table with 2 columns: 'empname' and 'deptname'. The table contains 6 rows of data.

	empname	deptname
1	Alice	HR
2	Bob	IT
3	Charlie	NULL
4	David	NULL
5	Emma	IT
6	NULL	Finance

-- Q2. Show employees and their department locations (include employees with no dept & depts with no employees).

```
select empname, deptname, location
from employees e
full join departments d
on e.deptid=d.deptid;
```



The screenshot shows a SQL query editor with the following text:

```
-- Q2. Show employees and their department locations (include employees with no dept & depts with no employees).
select empname, deptname, location
from employees e
full join departments d
on e.deptid=d.deptid;
```

Below the editor, the 'Results' tab is active, displaying a table with 3 columns: 'empname', 'deptname', and 'location'. The table contains 6 rows of data.

	empname	deptname	location
1	Alice	HR	Chennai
2	Bob	IT	Bangalore
3	Charlie	NULL	NULL
4	David	NULL	NULL
5	Emma	IT	Bangalore
6	NULL	Finance	Hyderabad

-- Q3. Find all employees who are not assigned to any department, and all departments that have no employees.
 -- (Hint: filter where DeptName IS NULL OR EmpName IS NULL).

```
select empname, deptname
from employees e
full join departments d
on e.deptid=d.deptid
where empname is null;
```

```
-- Q3. Find all employees who are not assigned to any department, and all departments that have no employees.
-- (Hint: filter where DeptName IS NULL OR EmpName IS NULL).

select empname, deptname
from employees e
full join departments d
on e.deptid=d.deptid
where empname is null;
```

	empname	deptname
1	NULL	Finance

```
select empname, deptname
from employees e
full join departments d
on e.deptid=d.deptid
where deptname is null;
```

```
select empname, deptname
from employees e
full join departments d
on e.deptid=d.deptid
where deptname is null;
```

	empname	deptname
1	Charlie	NULL
2	David	NULL

-- can give both as once by or

```
select empname, deptname
from employees e
full join departments d
on e.deptid=d.deptid
where empname is null or deptname is null;
```

```
-- can give both as once by or

select empname, deptname
from employees e
full join departments d
on e.deptid=d.deptid
where empname is null or deptname is null;
```

	empname	deptname
1	Charlie	NULL
2	David	NULL
3	NULL	Finance

-- Q4. Display all departments and employees, even if there is no match.
Show: EmpName, DeptName, Location.

```
select empname, deptname, location
from employees e
full join departments d
on e.deptid=d.deptid;
```

```
-- Q4. Display all departments and employees, even if there is no match. Show: EmpName, DeptName, Location.
select empname, deptname, location
from employees e
full join departments d
on e.deptid=d.deptid;
```

91 %

	empname	deptname	location
1	Alice	HR	Chennai
2	Bob	IT	Bangalore
3	Charlie	NULL	NULL
4	David	NULL	NULL
5	Emma	IT	Bangalore
6	NULL	Finance	Hyderabad

-- Q5. Count how many employees exist in each department, but also show departments with zero employees and employees with no department.
-- (Hint: Use GROUP BY DeptName with FULL OUTER JOIN).

```
select deptname, count(empname) as totalemp
from departments d
full join employees e
on d.deptid=e.deptid
group by deptname;
```

```
-- Q5. Count how many employees exist in each department, but also show departments with zero employees and employees with no department.
-- (Hint: Use GROUP BY DeptName with FULL OUTER JOIN).
select deptname, count(empname) as totalemp
from departments d
full join employees e
on d.deptid=e.deptid
group by deptname;
```

91 %

	deptname	totalemp
1	NULL	2
2	Finance	0
3	HR	1
4	IT	2

◆ What is CROSS JOIN?

- A **CROSS JOIN** returns the **Cartesian product** of two tables.
- This means **every row from the first table** is joined with **every row from the second table**.
- It does **not require any condition (ON clause)**.

🔗 Formula:

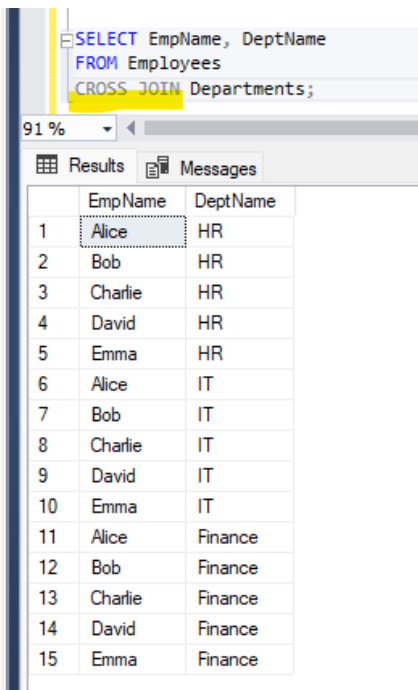
If Table A has **m rows** and Table B has **n rows**,
then **CROSS JOIN = $m \times n$ rows**.

Syntax:

```
SELECT columns
FROM TableA
CROSS JOIN TableB;
```

-- Q1. Show all possible employee-department combinations (basic CROSS JOIN).

```
SELECT EmpName, DeptName
FROM Employees
CROSS JOIN Departments;
```



The screenshot shows a SQL query window with the following text: `SELECT EmpName, DeptName FROM Employees CROSS JOIN Departments;`. Below the query, there is a 'Results' tab showing a table with two columns: 'EmpName' and 'DeptName'. The table contains 15 rows, representing all possible combinations of 5 employees and 3 departments.

	EmpName	DeptName
1	Alice	HR
2	Bob	HR
3	Charlie	HR
4	David	HR
5	Emma	HR
6	Alice	IT
7	Bob	IT
8	Charlie	IT
9	David	IT
10	Emma	IT
11	Alice	Finance
12	Bob	Finance
13	Charlie	Finance
14	David	Finance
15	Emma	Finance

employees table = 5 emp name

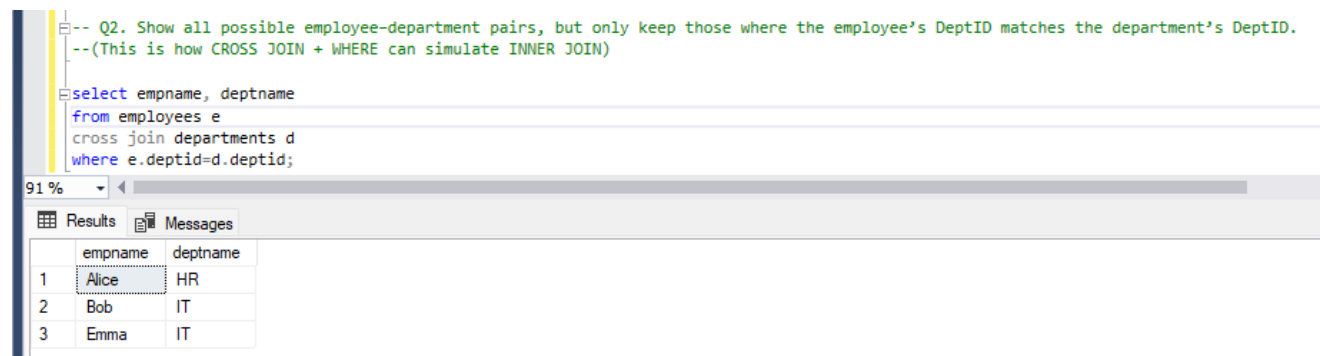
Department table = 3 department name

$\text{Emp} * \text{dept} = 5 * 3 = 15$

every employee is paired with every department, regardless of actual relation.

-- Q2. Show all possible employee-department pairs, but only keep those where the employee's DeptID matches the department's DeptID.
--(This is how CROSS JOIN + WHERE can simulate INNER JOIN)

```
select empname, deptname
from employees e
cross join departments d
where e.deptid=d.deptid;
```



Cross join + where = inner join

CROSS JOIN + WHERE (with matching condition) = **INNER JOIN**

Why?

CROSS JOIN → produces **all possible combinations** (Employees × Departments).

WHERE e.DeptID = d.DeptID → keeps only the rows where the keys match.

That's the **definition of INNER JOIN**.

In **real-world SQL**, you should **always use INNER JOIN** instead of writing CROSS JOIN + WHERE.

- INNER JOIN is **clearer** → anyone reading your query immediately knows the intention.
 - CROSS JOIN + WHERE is **confusing** → it first creates a big cartesian product and then filters, which is harder to read and not as efficient.
-
- Which query shows **actual data** (real matches)?
 - Which shows **all possibilities** (even if not real)?

The key takeaway:

- **INNER JOIN** → shows **real relationships**.
- **CROSS JOIN** → shows **all possible combinations** (whether they exist or not).

Q3. Find employees who could possibly belong to Finance (using CROSS JOIN, not INNER). Hint: CROSS JOIN gives you all pairs first, then you filter

```
select empname, deptname
from employees e
cross join departments d
where deptname='Finance';
```

```
-- Q3. Find employees who could possibly belong to Finance (using CROSS JOIN, not INNER).
-- Hint: CROSS JOIN gives you all pairs first, then you filter

select empname, deptname
from employees e
cross join departments d
where deptname='Finance';
```

91 %

Results Messages

	empname	deptname
1	Alice	Finance
2	Bob	Finance
3	Charlie	Finance
4	David	Finance
5	Emma	Finance

Q4. Show every employee paired with every possible location of departments.

```
select empname, location
from employees e
cross join departments d;
```

```
-- Q4. Show every employee paired with every possible location of departments.

select empname, location
from employees e
cross join departments d;
```

91 %

Results Messages

	empname	location
1	Alice	Chennai
2	Bob	Chennai
3	Charlie	Chennai
4	David	Chennai
5	Emma	Chennai
6	Alice	Bangalore
7	Bob	Bangalore
8	Charlie	Bangalore
9	David	Bangalore
10	Emma	Bangalore
11	Alice	Hyderabad
12	Bob	Hyderabad
13	Charlie	Hyderabad
14	David	Hyderabad
15	Emma	Hyderabad

-- Q5. Suppose every employee can attend any department's training. List all possible employee-training pairs. --(Think of CROSS JOIN as generating "all possible scenarios").

```
SELECT EmpName, DeptName AS TrainingDepartment
FROM Employees E
CROSS JOIN Departments D;
```

```
-- Q5. Suppose every employee can attend any department's training. List all possible employee-training pairs.
--(Think of CROSS JOIN as generating "all possible scenarios").

SELECT EmpName, DeptName AS TrainingDepartment
FROM Employees E
CROSS JOIN Departments D;
```

91 %

	EmpName	TrainingDepartment
1	Alice	HR
2	Bob	HR
3	Charlie	HR
4	David	HR
5	Emma	HR
6	Alice	IT
7	Bob	IT
8	Charlie	IT
9	David	IT
10	Emma	IT
11	Alice	Finance
12	Bob	Finance
13	Charlie	Finance
14	David	Finance
15	Emma	Finance

-- Q6. Assign every employee a potential project in each department (hypothetical mapping).

```
SELECT EmpName, DeptName,
       CONCAT(EmpName, '_Project_', DeptName) AS ProjectCode
FROM Employees E
CROSS JOIN Departments D;
```

Creates unique project codes like: Alice_Project_HR, Alice_Project_IT, Alice_Project_Finance, etc.

```
-- Q6. Assign every employee a potential project in each department (hypothetical mapping).

SELECT EmpName, DeptName,
       CONCAT(EmpName, '_Project_', DeptName) AS ProjectCode
FROM Employees E
CROSS JOIN Departments D;
```

91 %

	EmpName	DeptName	ProjectCode
1	Alice	HR	Alice_Project_HR
2	Bob	HR	Bob_Project_HR
3	Charlie	HR	Charlie_Project_HR
4	David	HR	David_Project_HR
5	Emma	HR	Emma_Project_HR
6	Alice	IT	Alice_Project_IT
7	Bob	IT	Bob_Project_IT
8	Charlie	IT	Charlie_Project_IT
9	David	IT	David_Project_IT
10	Emma	IT	Emma_Project_IT
11	Alice	Finance	Alice_Project_Finance
12	Bob	Finance	Bob_Project_Finance
13	Charlie	Finance	Charlie_Project_Finance
14	David	Finance	David_Project_Finance
15	Emma	Finance	Emma_Project_Finance

⚡ Key idea:

CROSS JOIN is rarely used in real business queries unless:

You want **all possible combinations** (like schedules, pairing employees with shifts, assigning tasks).

Or you filter later (`WHERE`) to simulate inner join-like scenarios.

SELF JOIN (Self-join = when a table joins with itself)

◆ What is a Self Join?

- A **self join** is a **regular join** but the table is joined with itself.
- Useful when rows in the **same table** have a relationship (like an employee reporting to a manager, or prerequisite relationships in a course list).

★ Example scenario with Employees table

Let's extend your **Employees** table by adding a `ManagerID` column:

EmpID EmpName DeptID ManagerID

1	Alice	1	NULL
2	Bob	2	1
3	Charlie	NULL	2
4	David	NULL	2
5	Emma	2	1

- Alice is a Manager (no manager above her, no `ManagerID`).
- Bob & Emma report to Alice.
- Charlie & David report to Bob.

Output:

Employee	Manager
Alice	NULL
Bob	Alice
Charlie	Bob
David	Bob
Emma	Alice

❏ Syntax for Self Join:

```
SELECT E.EmpName AS Employee, M.EmpName AS Manager
FROM Employees E
LEFT JOIN Employees M
ON E.ManagerID = M.EmpID;
```

real-life style dataset for self join practice:

-- for self join practice will create new table as manager

```
create table managers(
managerId int primary key,
managerName varchar(20),
department varchar(10),
location varchar(25),
reportsto int null);    -- refers to another manager ID
```

```
insert into managers (managerId, managerName, department, location, reportsto)
values
```

```
(1, 'Alice', 'HR', 'Chennai', NULL),    -- Alice is top-level
(2, 'Bob', 'IT', 'Bangalore', 1),       -- Bob reports to Alice
(3, 'Charlie', 'Finance', 'Hyderabad', 1), -- Charlie reports to Alice
(4, 'David', 'IT', 'Chennai', 2),       -- David reports to Bob
(5, 'Emma', 'Finance', 'Bangalore', 3); -- Emma reports to Charlie
```

```
select * from managers;
```

```
-- for self join practice will create new table as manager

create table managers(
managerId int primary key,
managerName varchar(20),
department varchar(10),
location varchar(25),
reportsto int null);    -- refers to another manager ID

insert into managers (managerId, managerName, department, location, reportsto)
values
(1, 'Alice', 'HR', 'Chennai', NULL),    -- Alice is top-level
(2, 'Bob', 'IT', 'Bangalore', 1),       -- Bob reports to Alice
(3, 'Charlie', 'Finance', 'Hyderabad', 1), -- Charlie reports to Alice
(4, 'David', 'IT', 'Chennai', 2),       -- David reports to Bob
(5, 'Emma', 'Finance', 'Bangalore', 3); -- Emma reports to Charlie

select * from managers;
```

90 %

Results Messages

	managerId	managerName	department	location	reportsto
1	1	Alice	HR	Chennai	NULL
2	2	Bob	IT	Bangalore	1
3	3	Charlie	Finance	Hyderabad	1
4	4	David	IT	Chennai	2
5	5	Emma	Finance	Bangalore	3

1 Normal SELECT

```
SELECT ManagerID, ManagerName, Report_To  
FROM Managers;
```

🔗 Output:

ManagerID	ManagerName	Report_To
-----------	-------------	-----------

1	Alice	NULL
2	Bob	1
3	Charlie	1
4	David	2
5	Emma	3

→ Here `Report_To` just shows **ID numbers** (not helpful for understanding who the boss is).

2 SELF JOIN

```
SELECT  
    E.ManagerName AS Employee,  
    M.ManagerName AS Boss  
FROM Managers E  
LEFT JOIN Managers M  
    ON E.Report_To = M.ManagerID;
```

🔗 Output:

Employee	Boss
----------	------

Alice	NULL
Bob	Alice
Charlie	Alice
David	Bob
Emma	Charlie

→ Now it's clear: **Bob reports to Alice, David reports to Bob, Emma reports to Charlie.**

✓ **Key takeaway:**

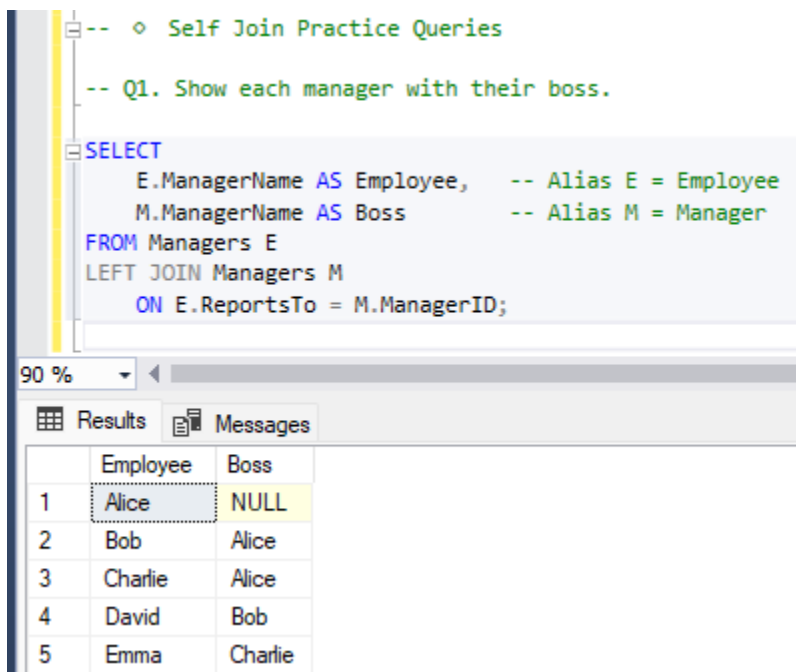
- Normal SELECT → Just IDs
- SELF JOIN → Translates IDs into **meaningful names** (relationships)
- or a **SELF JOIN**, we are joining the same table with itself.

Since SQL doesn't allow you to use the same table name twice directly, we give **aliases** (short names) to treat them like two separate tables.

-- ♦ Self Join Practice Queries

Q1. Show each manager with their boss.

```
SELECT
    E.ManagerName AS Employee, -- Alias E = Employee
    M.ManagerName AS Boss      -- Alias M = Manager
FROM Managers E
LEFT JOIN Managers M
ON E.ReportsTo = M.ManagerID;
```



The screenshot shows a SQL query editor with the following text:

```
-- ♦ Self Join Practice Queries

-- Q1. Show each manager with their boss.

SELECT
    E.ManagerName AS Employee, -- Alias E = Employee
    M.ManagerName AS Boss      -- Alias M = Manager
FROM Managers E
LEFT JOIN Managers M
ON E.ReportsTo = M.ManagerID;
```

Below the query editor, the 'Results' tab is active, displaying a table with 5 rows and 2 columns: 'Employee' and 'Boss'.

	Employee	Boss
1	Alice	NULL
2	Bob	Alice
3	Charlie	Alice
4	David	Bob
5	Emma	Charlie

```
-- Q2. Find all employees who report to 'Alice'.  
-- (Hint: Self join, filter Boss = 'Alice')
```

```
SELECT  
    E.ManagerName AS Employee,  
    M.ManagerName AS Boss  
FROM Managers E  
LEFT JOIN Managers M  
    ON E.ReportsTo = M.ManagerID  
    where M.ManagerName='Alice';
```

```
-- Q2. Find all employees who report to 'Alice'.  
-- (Hint: Self join, filter Boss = 'Alice')  
  
SELECT  
    E.ManagerName AS Employee,  
    M.ManagerName AS Boss  
FROM Managers E  
LEFT JOIN Managers M  
    ON E.ReportsTo = M.ManagerID  
    where M.ManagerName='Alice';
```

90 %

	Employee	Boss
1	Bob	Alice
2	Charlie	Alice

```
-- Q3. Find all employees who are managers themselves (i.e., others report to them).
```

```
SELECT DISTINCT M.ManagerName AS Manager  
FROM Managers E  
JOIN Managers M  
    ON E.ReportsTo = M.ManagerID;
```

```
-- Q3. Find all employees who are managers themselves (i.e., others report to them).  
  
SELECT DISTINCT M.ManagerName AS Manager  
FROM Managers E  
JOIN Managers M  
    ON E.ReportsTo = M.ManagerID;
```

90 %

	Manager
1	Alice
2	Bob
3	Charlie

DISTINCT is used in SQL to **remove duplicate rows** from the result set.

✦ Example without DISTINCT

Suppose we run this query (Q3 from above):

```
SELECT M.ManagerName AS Manager
FROM Managers E
JOIN Managers M
    ON E.Report_To = M.ManagerID;
```

🔗 Output:

Manager

Alice

Alice

Bob

Charlie

Notice: **Alice appears twice** (because both Bob & Charlie report to her).

Example with DISTINCT

```
SELECT DISTINCT M.ManagerName AS Manager
FROM Managers E
JOIN Managers M
    ON E.Report_To = M.ManagerID;
```

🔗 Output:

Manager

Alice

Bob

Charlie

✓ DISTINCT removed the duplicate Alice.

- **DISTINCT** → Only removes duplicates, no aggregation.
- **GROUP BY** → Groups rows, allows you to use aggregates (COUNT, SUM, MAX, etc.).
- If no aggregates, GROUP BY = same as DISTINCT.

-- Q4.List employees and their "grandboss" (manager's manager).
-- (This will need a double self join → Employees → Manager → Manager's Manager)

SELECT

E.ManagerName AS Employee,
M.ManagerName AS Boss,
GM.ManagerName AS GrandBoss

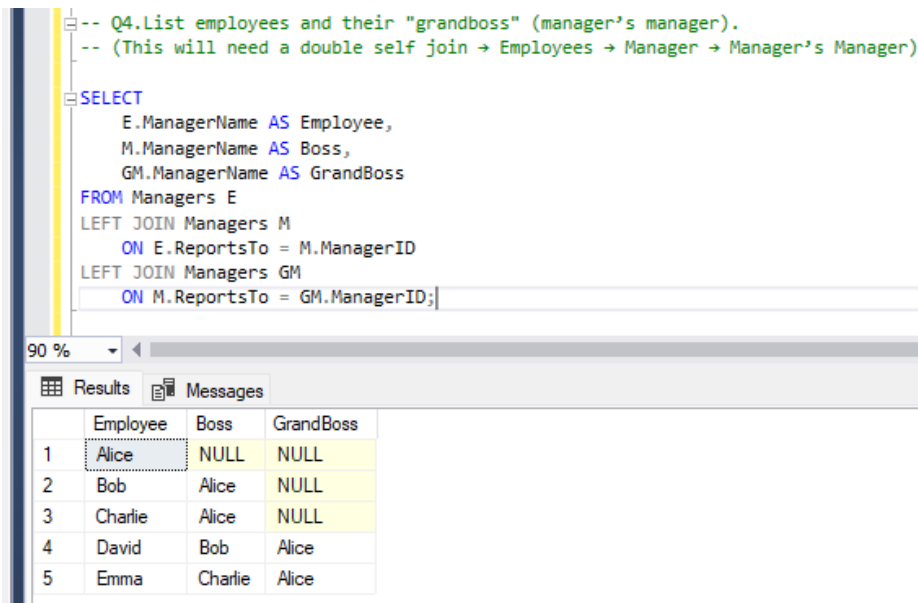
FROM Managers E

LEFT JOIN Managers M

ON E.ReportsTo = M.ManagerID

LEFT JOIN Managers GM

ON M.ReportsTo = GM.ManagerID;



The screenshot shows a SQL query editor with the following query:

```
-- Q4.List employees and their "grandboss" (manager's manager).  
-- (This will need a double self join → Employees → Manager → Manager's Manager)  
  
SELECT  
    E.ManagerName AS Employee,  
    M.ManagerName AS Boss,  
    GM.ManagerName AS GrandBoss  
FROM Managers E  
LEFT JOIN Managers M  
    ON E.ReportsTo = M.ManagerID  
LEFT JOIN Managers GM  
    ON M.ReportsTo = GM.ManagerID;
```

Below the query editor, the results are displayed in a table with columns: Employee, Boss, and GrandBoss.

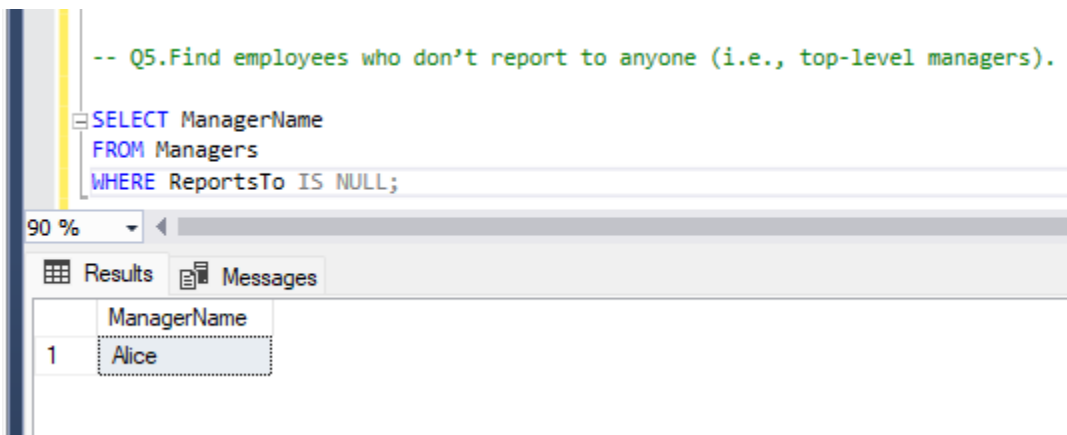
	Employee	Boss	GrandBoss
1	Alice	NULL	NULL
2	Bob	Alice	NULL
3	Charlie	Alice	NULL
4	David	Bob	Alice
5	Emma	Charlie	Alice

-- Q5.Find employees who don't report to anyone (i.e., top-level managers).

SELECT ManagerName

FROM Managers

WHERE ReportsTo IS NULL;



The screenshot shows a SQL query editor with the following query:

```
-- Q5.Find employees who don't report to anyone (i.e., top-level managers).  
  
SELECT ManagerName  
FROM Managers  
WHERE ReportsTo IS NULL;
```

Below the query editor, the results are displayed in a table with columns: ManagerName.

	ManagerName
1	Alice

1. **INNER JOIN**
 2. **LEFT JOIN (LEFT OUTER JOIN)**
 3. **RIGHT JOIN (RIGHT OUTER JOIN)**
 4. **FULL JOIN (FULL OUTER JOIN)**
 5. **CROSS JOIN**
 6. **SELF JOIN**
-

⚡ optional to know

(not always in interviews, but good for strong knowledge):

7. **NATURAL JOIN** (*not supported in SQL Server, but exists in MySQL, Oracle, PostgreSQL — joins automatically on columns with the same name*)
8. **USING clause** (instead of **ON**, if both tables have same column name) → not in SQL Server, but in other RDBMS.
9. **Equi Join vs Non-Equi Join**
 - *Equi Join* → join condition uses = (all joins we did so far).
 - *Non-Equi Join* → join condition uses <, >, BETWEEN, etc. (rare but possible).

🔹 1. Equi Join

- Uses an **equality operator (=)** in the join condition.
- Almost all joins we practiced so far (INNER, LEFT, RIGHT, FULL) are **Equi Joins** because we matched `DeptID = DeptID`.

🔗 Example:

```
SELECT EmpName, DeptName
FROM Employees E
INNER JOIN Departments D
    ON E.DeptID = D.DeptID;
```

✓ Output (matches on equality):

EmpName DeptName

Alice	HR
Bob	IT
Emma	IT

2. Non-Equi Join

- Uses operators **other than =** (like <, >, BETWEEN, !=) in the join condition.
- Useful when relationship is based on **range** instead of exact match.

Example Dataset

Employees

EmpID	EmpName	Salary
1	Alice	6000
2	Bob	4000
3	Charlie	2000

SalaryGrades

Grade	MinSalary	MaxSalary
A	5000	7000
B	3000	4999
C	1000	2999

Non-Equi Join Query:

```
SELECT E.EmpName, E.Salary, S.Grade
FROM Employees E
JOIN SalaryGrades S
      ON E.Salary BETWEEN S.MinSalary AND S.MaxSalary;
```

Output:

EmpName	Salary	Grade
Alice	6000	A
Bob	4000	B
Charlie	2000	C

Key Difference

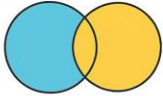
- **Equi Join** → strict equality (=).
- **Non-Equi Join** → range or other comparison operators (<, >, BETWEEN, !=).

SQL JOINS

SQL JOIN is used to combine rows from two or more tables based on related columns between them.

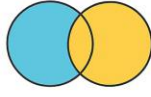
SQL JOIN TYPES

Inner Join



```
SELECT columns FROM TableA  
INNER JOIN TableB  
ON TableA.column = TableB.column
```

Left Join (Left Outer Join)



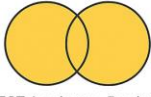
```
SELECT columns FROM TableA  
LEFT JOIN TableB  
ON TableA.column = TabB.column
```

Right Join (Right Outer Join)



```
SELECT columns FROM TableA  
FULL JOIN TableB  
ON TableA.column = TableB.column
```

Cross Join



```
SELECT A.columns, B.columns  
FROM TableA A JOIN  
TableA.column = Tabl.column
```

SQL JOINING

Equi Join

- Uses the equality (=) operator in the join condition

```
SELECT columns  
FROM TableA JOIN 'TableA'  
ON TableA.column = Tabl.column
```

Non-Equi Join

- Uses operators other than the equality (=) operator in the join condition

```
SELECT columns  
FROM TableA JOIN 'TableA'  
ON TableA.column > B.column
```

👉 **Joins are not limited to 2 tables.**

You can join **3, 4, or even more tables** in a single query. As long as there are relationships (common columns/keys), you can chain joins together.

For joining multiple tables : query is down – output is attached screenshot

```
select * from employees1;  
select * from departments1;  
select * from locations1;
```

	EmpID	EmpName	DeptID
1	1	Alice	1
2	2	Bob	2
3	3	Charlie	NULL
4	4	David	3
5	5	Emma	2

	DeptID	DeptName	LocationID
1	1	HR	101
2	2	IT	102
3	3	Finance	103

	LocationID	City	Country
1	101	Chennai	India
2	102	Bangalore	India
3	103	Hyderab...	India
4	104	Delhi	India

```
-- MULTI TABLE Joins
```

```
-- Tables for Practice
```

```
-- 1. Employees
```

```
CREATE TABLE Employees1 (  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50),  
    DeptID INT);
```

```
INSERT INTO Employees1 (EmpID, EmpName, DeptID) VALUES  
(1, 'Alice', 1),  
(2, 'Bob', 2),  
(3, 'Charlie', NULL),  
(4, 'David', 3),  
(5, 'Emma', 2);
```

```
-- 2. Departments
```

```
CREATE TABLE Departments1 (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50),  
    LocationID INT);
```

```
INSERT INTO Departments1 (DeptID, DeptName, LocationID) VALUES  
(1, 'HR', 101),  
(2, 'IT', 102),  
(3, 'Finance', 103);
```

```
-- 3. Locations
```

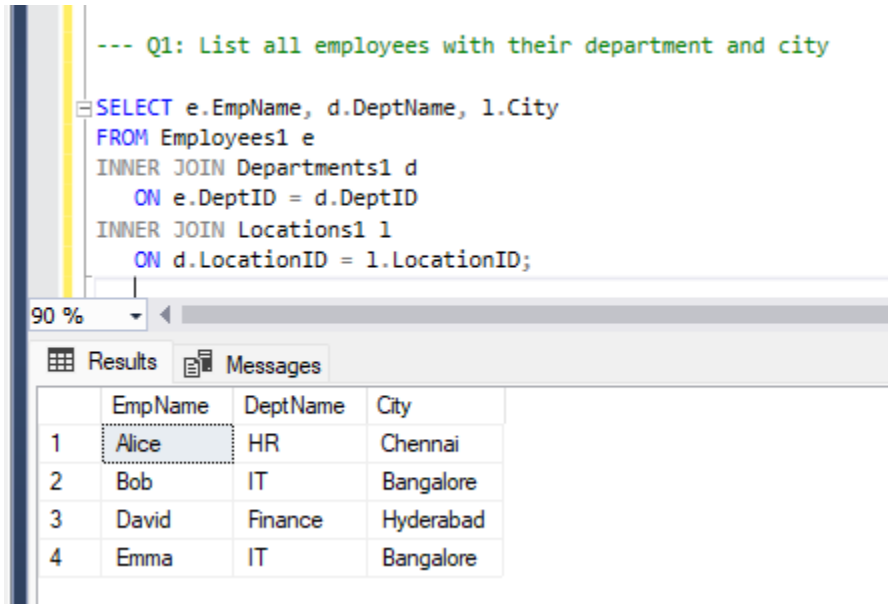
```
CREATE TABLE Locations1 (  
    LocationID INT PRIMARY KEY,  
    City VARCHAR(50),  
    Country VARCHAR(50));
```

```
INSERT INTO Locations1 (LocationID, City, Country) VALUES  
(101, 'Chennai', 'India'),  
(102, 'Bangalore', 'India'),  
(103, 'Hyderabad', 'India'),  
(104, 'Delhi', 'India');
```

```
select * from employees1;  
select * from departments1;  
select * from locations1;
```

--- Q1: List all employees with their department and city

```
SELECT e.EmpName, d.DeptName, l.City
FROM Employees1 e
INNER JOIN Departments1 d
    ON e.DeptID = d.DeptID
INNER JOIN Locations1 l
    ON d.LocationID = l.LocationID;
```



The screenshot shows a SQL IDE window with a query editor and a results pane. The query editor contains the same SQL query as above. The results pane shows a table with 4 rows and 4 columns: EmpName, DeptName, City, and an implicit ID column. The data is as follows:

	EmpName	DeptName	City
1	Alice	HR	Chennai
2	Bob	IT	Bangalore
3	David	Finance	Hyderabad
4	Emma	IT	Bangalore

-- here we have 3 table - employees1, departments1, location1
-- deptID is common between employees and department table
-- LocationID is common between department and location table
-- we want employees , department and city in output-- where all 3 column is
in different 3 tables
-- as usual we will join bt will use joins twice

✓ You can keep extending

```
FROM A
JOIN B ON ...
JOIN C ON ...
JOIN D ON ...
```

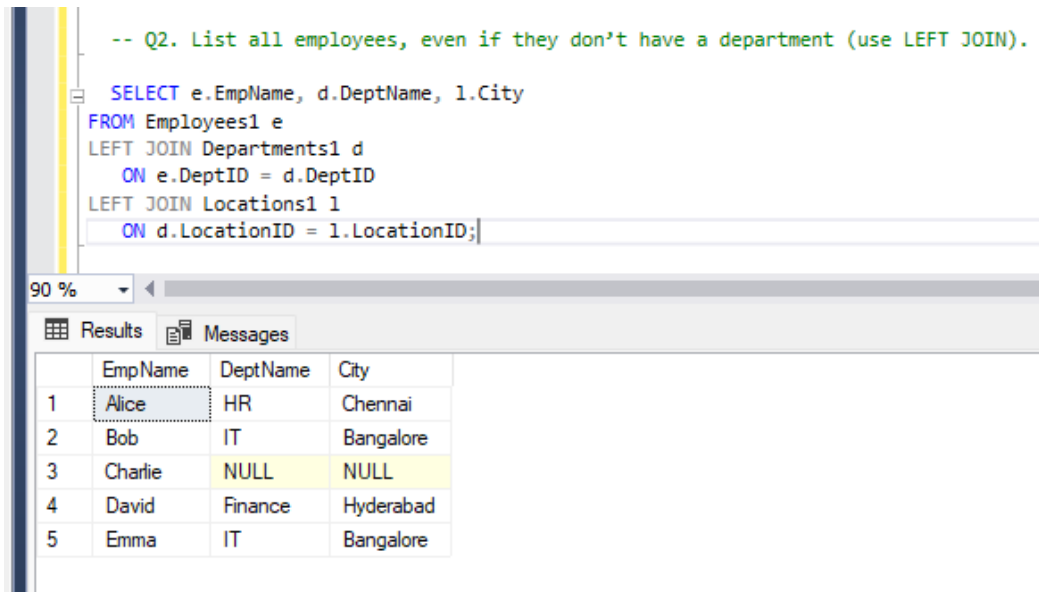
There's no strict limit (other than performance and readability). In real projects, joining **4–6 tables** is common.

✂ Key Tip:

- Always join logically (foreign key ↔ primary key).
- Use **aliases** (e, d, l) to avoid confusion.

Q2. List all employees, even if they don't have a department (use LEFT JOIN).

```
SELECT e.EmpName, d.DeptName, l.City
FROM Employees1 e
LEFT JOIN Departments1 d
  ON e.DeptID = d.DeptID
LEFT JOIN Locations1 l
  ON d.LocationID = l.LocationID;
```



The screenshot shows a SQL query editor with the following text:

```
-- Q2. List all employees, even if they don't have a department (use LEFT JOIN).

SELECT e.EmpName, d.DeptName, l.City
FROM Employees1 e
LEFT JOIN Departments1 d
  ON e.DeptID = d.DeptID
LEFT JOIN Locations1 l
  ON d.LocationID = l.LocationID;
```

Below the query editor, there is a tab labeled "Results" which displays the following table:

	EmpName	DeptName	City
1	Alice	HR	Chennai
2	Bob	IT	Bangalore
3	Charlie	NULL	NULL
4	David	Finance	Hyderabad
5	Emma	IT	Bangalore

-- Q3. Show all departments with their employees and city (even if no employee exists).

```
select d.deptname, e.empname, l.city
from departments1 d
left join employees1 e
  on e.deptid=d.deptid
left join locations1 l
  on d.locationid=l.locationid;
```

-- or

```
select d.deptname, e.empname, l.city
from departments1 d
left join employees1 e
  on e.deptid=d.deptid
inner join locations1 l
  on d.locationid=l.locationid;
```

```
-- Q3. Show all departments with their employees and city (even if no employee exists).

select d.deptname, e.empname, l.city
from departments1 d
left join employees1 e
on e.deptid=d.deptid
left join locations1 l
on d.locationid=l.locationid;

-- or

select d.deptname, e.empname, l.city
from departments1 d
left join employees1 e
on e.deptid=d.deptid
inner join locations1 l
on d.locationid=l.locationid;
```

90 %

Results Messages

	deptname	empname	city
1	HR	Alice	Chennai
2	IT	Bob	Bangalore
3	IT	Emma	Bangalore
4	Finance	David	Hyderabad

	deptname	empname	city
1	HR	Alice	Chennai
2	IT	Bob	Bangalore
3	IT	Emma	Bangalore
4	Finance	David	Hyderab...

-- Q4. Count how many employees are in each city.

```
select l.city, count(empid) as total
from locations1 l
left join departments1 d
on l.locationid=d.locationid
left join employees1 e
on e.deptid=d.deptid
group by l.city;
```

```
-- Q4. Count how many employees are in each city.

select l.city, count(empid) as total
from locations1 l
left join departments1 d
on l.locationid=d.locationid
left join employees1 e
on e.deptid=d.deptid
group by l.city;
```

90 %

Results Messages

	city	total
1	Bangalore	2
2	Chennai	1
3	Delhi	0
4	Hyderabad	1

-- Dataset 2: Students, Courses, Enrollments

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(50),  
    Age INT  
);
```

```
CREATE TABLE Courses (  
    CourseID INT PRIMARY KEY,  
    CourseName VARCHAR(50),  
    Credits INT  
);
```

```
CREATE TABLE Enrollments (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,  
    CourseID INT,  
    Grade CHAR(1),  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),  
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)  
);
```

-- Insert Students

```
INSERT INTO Students VALUES  
(1, 'Ravi', 20),  
(2, 'Priya', 21),  
(3, 'Karthik', 22),  
(4, 'Meena', 20);
```

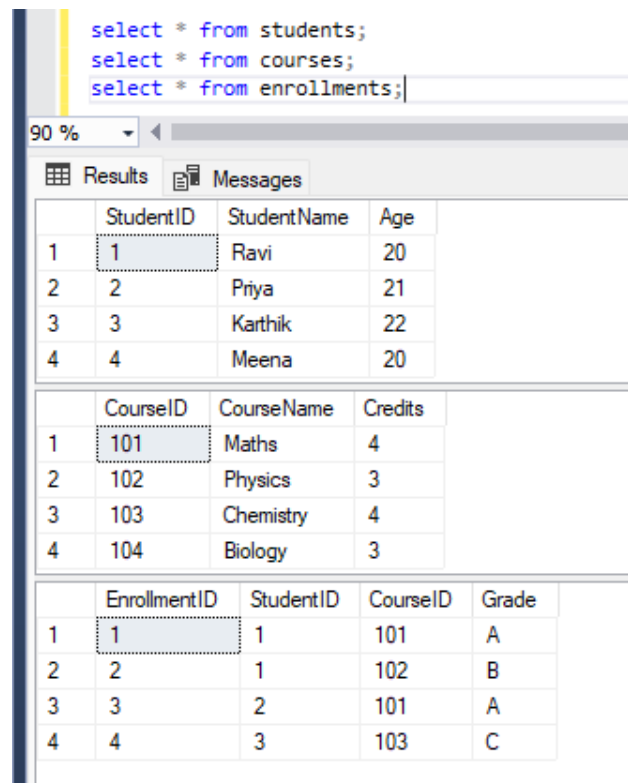
-- Insert Courses

```
INSERT INTO Courses VALUES  
(101, 'Maths', 4),  
(102, 'Physics', 3),  
(103, 'Chemistry', 4),  
(104, 'Biology', 3);
```

-- Insert Enrollments

```
INSERT INTO Enrollments VALUES  
(1, 1, 101, 'A'),  
(2, 1, 102, 'B'),  
(3, 2, 101, 'A'),  
(4, 3, 103, 'C');
```

```
select * from students;  
select * from courses;  
select * from enrollments;
```



The screenshot shows a database query interface with the following SQL commands and results:

```
select * from students;  
select * from courses;  
select * from enrollments;
```

Students Table:

	StudentID	StudentName	Age
1	1	Ravi	20
2	2	Priya	21
3	3	Karthik	22
4	4	Meena	20

Courses Table:

	CourseID	CourseName	Credits
1	101	Maths	4
2	102	Physics	3
3	103	Chemistry	4
4	104	Biology	3

Enrollments Table:

	EnrollmentID	StudentID	CourseID	Grade
1	1	1	101	A
2	2	1	102	B
3	3	2	101	A
4	4	3	103	C

-- Q1. List all students with the courses they are enrolled in.

```
select s.studentName, c.CourseName
from students s
left join enrollments e
on s.studentId=e.studentId
left join courses c
on e.courseid=c.courseid;
```

-- Q1. List all students with the courses they are enrolled in.

```
select s.studentName, c.CourseName
from students s
left join enrollments e
on s.studentId=e.studentId
left join courses c
on e.courseid=c.courseid;
```

90 %

Results Messages

	studentName	CourseName
1	Ravi	Maths
2	Ravi	Physics
3	Priya	Maths
4	Karthik	Chemistry
5	Meena	NULL

-- Q2. Find students who are not enrolled in any course.

```
select s.studentName,
c.CourseName
from students s
left join enrollments e
on s.studentId=e.studentId
left join courses c
on e.courseid=c.courseid
where courseName is null;
```

-- or

```
SELECT s.StudentName
FROM Students s
LEFT JOIN Enrollments e
ON s.StudentID = e.StudentID
WHERE e.StudentID IS NULL;
```

-- Q2. Find students who are not enrolled in any course.

```
select s.studentName, c.CourseName
from students s
left join enrollments e
on s.studentId=e.studentId
left join courses c
on e.courseid=c.courseid
where courseName is null;

-- or

SELECT s.StudentName
FROM Students s
LEFT JOIN Enrollments e
ON s.StudentID = e.StudentID
WHERE e.StudentID IS NULL;
```

90 %

Results Messages

	studentName	CourseName
1	Meena	NULL

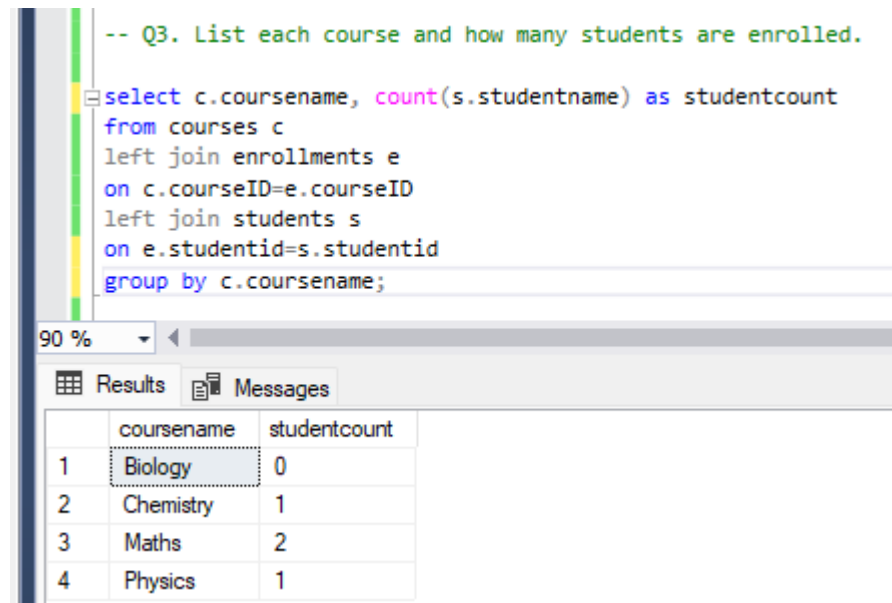
	StudentName
1	Meena

-- Q3. List each course and how many students are enrolled.

```
select c.coursename, count(s.studentname) as studentcount
from courses c
left join enrollments e
on c.courseID=e.courseID
left join students s
on e.studentid=s.studentid
group by c.coursename;
```

☞ Rule of thumb:

- If your SELECT list has **only aggregates** → no GROUP BY required.
- If your SELECT list has **columns + aggregates** → you **must** group by the non-aggregated columns.



The screenshot shows a SQL IDE with a query editor and a results pane. The query editor contains the following SQL code:

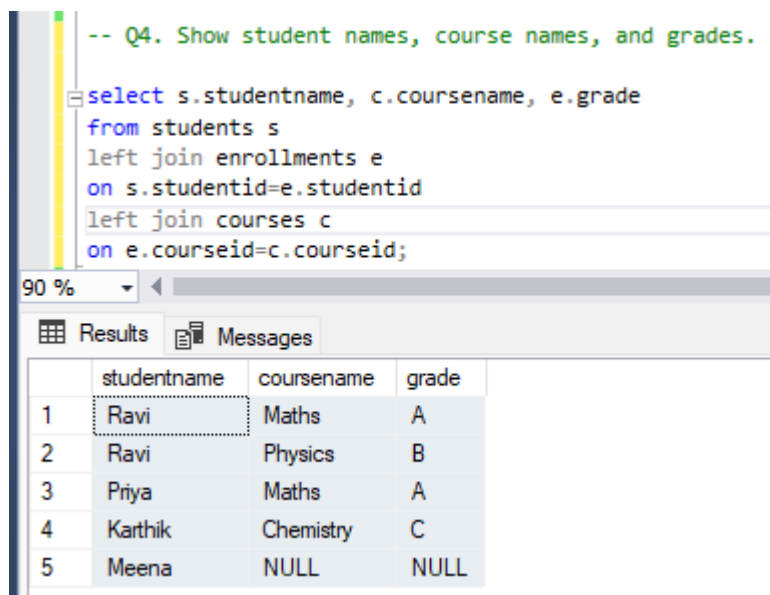
```
-- Q3. List each course and how many students are enrolled.
select c.coursename, count(s.studentname) as studentcount
from courses c
left join enrollments e
on c.courseID=e.courseID
left join students s
on e.studentid=s.studentid
group by c.coursename;
```

The results pane shows a table with two columns: 'coursename' and 'studentcount'. The table contains four rows of data:

	coursename	studentcount
1	Biology	0
2	Chemistry	1
3	Maths	2
4	Physics	1

-- Q4. Show student names, course names, and grades.

```
select s.studentname, c.coursename, e.grade
from students s
left join enrollments e
on s.studentid=e.studentid
left join courses c
on e.courseid=c.courseid;
```



The screenshot shows a SQL IDE with a query editor and a results pane. The query editor contains the following SQL code:

```
-- Q4. Show student names, course names, and grades.
select s.studentname, c.coursename, e.grade
from students s
left join enrollments e
on s.studentid=e.studentid
left join courses c
on e.courseid=c.courseid;
```

The results pane shows a table with three columns: 'studentname', 'coursename', and 'grade'. The table contains five rows of data:

	studentname	coursename	grade
1	Ravi	Maths	A
2	Ravi	Physics	B
3	Priya	Maths	A
4	Karthik	Chemistry	C
5	Meena	NULL	NULL

-- Q5. Find courses that no student has enrolled in.

```
select c.coursename,  
s.studentname  
from courses c  
left join enrollments e  
on c.courseID=e.courseID  
left join students s  
on e.studentid=s.studentid  
where s.studentname is null;
```

--or

```
select c.coursename  
from courses c  
left join enrollments e  
on c.courseID = e.courseID  
where e.studentid is null;
```

```
-- Q5. Find courses that no student has enrolled in.  
select c.coursename, s.studentname  
from courses c  
left join enrollments e  
on c.courseID=e.courseID  
left join students s  
on e.studentid=s.studentid  
where s.studentname is null;  
  
--or  
select c.coursename  
from courses c  
left join enrollments e on c.courseID = e.courseID  
where e.studentid is null;
```

0 %

Results Messages

	coursename	studentname
1	Biology	NULL

-- Q6. List each student with the total number of courses they are enrolled in

```
select s.studentname, count(c.courseid) as total  
from students s  
left join enrollments e  
on s.studentid=e.studentid  
left join courses c  
on e.courseid=c.courseid  
group by studentname;
```

```
-- Q6. List each student with the total number of courses they are enrolled in  
select s.studentname, count(c.courseid) as total  
from students s  
left join enrollments e  
on s.studentid=e.studentid  
left join courses c  
on e.courseid=c.courseid  
group by studentname;
```

90 %

Results Messages

	studentname	total
1	Karthik	1
2	Meena	0
3	Priya	1
4	Ravi	2

-- Dataset 2: Orders, Customers, Products

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    CustomerName VARCHAR(50),  
    City VARCHAR(50)  
);
```

```
CREATE TABLE Products (  
    ProductID INT PRIMARY KEY,  
    ProductName VARCHAR(50),  
    Price DECIMAL(10,2)  
);
```

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    ProductID INT,  
    Quantity INT,  
    OrderDate DATE,  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),  
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)  
);
```

-- Insert Customers

```
INSERT INTO Customers VALUES  
(1, 'Suresh', 'Chennai'),  
(2, 'Anita', 'Bangalore'),  
(3, 'Rajesh', 'Hyderabad'),  
(4, 'Divya', 'Chennai');
```

-- Insert Products

```
INSERT INTO Products VALUES  
(101, 'Laptop', 55000),  
(102, 'Mobile', 20000),  
(103, 'Tablet', 30000),  
(104, 'Headphones', 2000);
```

-- Insert Orders

```
INSERT INTO Orders VALUES  
(1, 1, 101, 1, '2025-08-01'),  
(2, 1, 104, 2, '2025-08-05'),  
(3, 2, 102, 1, '2025-08-10'),  
(4, 3, 103, 1, '2025-08-12');
```

```
select * from customers;  
select * from products;  
select * from orders;
```

```
select * from customers;  
select * from products;  
select * from orders;
```

CustomerID	CustomerName	City
1	Suresh	Chennai
2	Anita	Bangalore
3	Rajesh	Hyderabad
4	Divya	Chennai

ProductID	ProductName	Price
101	Laptop	55000.00
102	Mobile	20000.00
103	Tablet	30000.00
104	Headphones	2000.00

OrderID	CustomerID	ProductID	Quantity	OrderDate
1	1	101	1	2025-08-01
2	1	104	2	2025-08-05
3	2	102	1	2025-08-10
4	3	103	1	2025-08-12

In SQL, if you just write **JOIN** (without saying LEFT, RIGHT, or FULL), it is automatically treated as **INNER JOIN**.

-- Sample Questions

-- Q1. List all orders with customer names and product names.

```
select c.customername, p.productname
from customers c
left join orders o
on c.customerid=o.customerid
inner join products p
on o.productid=p.productid;
```

```
-- Q1. List all orders with customer names and product names.
select c.customername, p.productname
from customers c
left join orders o
on c.customerid=o.customerid
inner join products p
on o.productid=p.productid;
```

90 %

	customername	productname
1	Suresh	Laptop
2	Suresh	Headphones
3	Anita	Mobile
4	Rajesh	Tablet

-- Q2. Show total sales amount per customer.

```
select c.customername,
sum(p.price) as totalamount
from customers c
left join orders o
on c.customerid=o.customerid
inner join products p
on o.productid=p.productid
group by c.customername
order by totalamount desc;
```

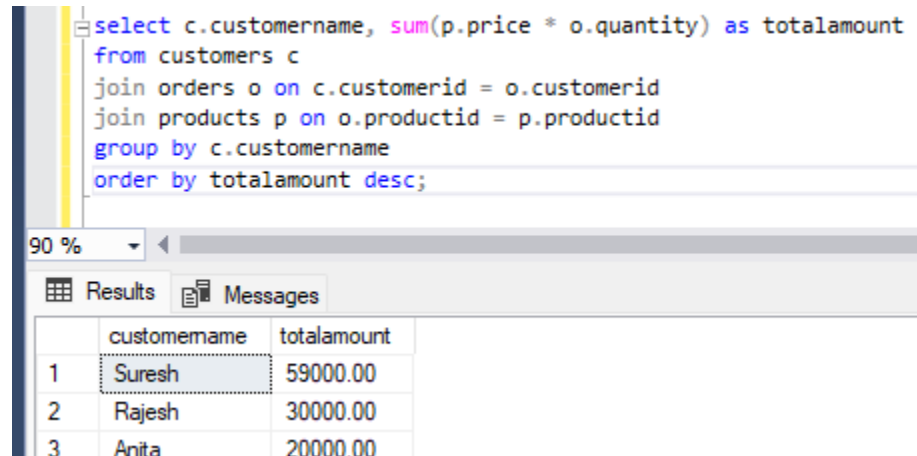
```
-- Q2. Show total sales amount per customer.
select c.customername, sum(p.price) as totalamount
from customers c
left join orders o
on c.customerid=o.customerid
inner join products p
on o.productid=p.productid
group by c.customername
order by totalamount desc;
```

90 %

	customername	totalamount
1	Suresh	57000.00
2	Rajesh	30000.00
3	Anita	20000.00

--OR

```
select c.customername, sum(p.price * o.quantity) as totalamount
from customers c
join orders o on c.customerid = o.customerid
join products p on o.productid = p.productid
group by c.customername
order by totalamount desc;
```



```
select c.customername, sum(p.price * o.quantity) as totalamount
from customers c
join orders o on c.customerid = o.customerid
join products p on o.productid = p.productid
group by c.customername
order by totalamount desc;
```

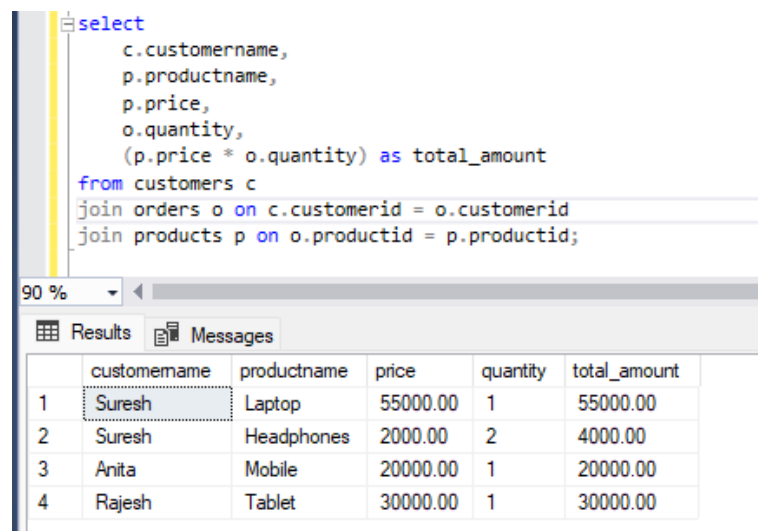
90 %

Results Messages

	customername	totalamount
1	Suresh	59000.00
2	Rajesh	30000.00
3	Anita	20000.00

--OR

```
select c.customername, p.productname, p.price, o.quantity,
(p.price * o.quantity) as total_amount
from customers c
join orders o on c.customerid = o.customerid
join products p on o.productid = p.productid;
```



```
select
c.customername,
p.productname,
p.price,
o.quantity,
(p.price * o.quantity) as total_amount
from customers c
join orders o on c.customerid = o.customerid
join products p on o.productid = p.productid;
```

90 %

Results Messages

	customername	productname	price	quantity	total_amount
1	Suresh	Laptop	55000.00	1	55000.00
2	Suresh	Headphones	2000.00	2	4000.00
3	Anita	Mobile	20000.00	1	20000.00
4	Rajesh	Tablet	30000.00	1	30000.00

-- Q3. Find customers who have never placed an order.

```
select c.customername
from customers c
left join orders o
on c.customerid=o.customerid
left join products p
on o.productid=p.productid
where p.productid is null;
```

-- or

```
select c.customername
from customers c
left join orders o
on c.customerid = o.customerid
where o.orderid is null;
```

-- Q3. Find customers who have never placed an order.

```
select c.customername
from customers c
left join orders o
on c.customerid=o.customerid
left join products p
on o.productid=p.productid
where p.productid is null;
```

-- or

```
select c.customername
from customers c
left join orders o on c.customerid = o.customerid
where o.orderid is null;
```

90 %

Results Messages

	customername
1	Divya

	customername
1	Divya

-- Q4. List top 3 most ordered products.

-- wrong answer

-- used MAX(quantity), which only shows the largest single order for that product.

-- We need the total quantity ordered per product.

```
select p.productname,
max(o.quantity) as most
from products p
left join orders o
on p.productid=o.productid
group by p.productname;
```

-- Q4. List top 3 most ordered products.

-- wrong answer

-- used MAX(quantity), which only shows the largest single order for that product.

-- We need the total quantity ordered per product.

```
select p.productname, max(o.quantity) as most
from products p
left join orders o
on p.productid=o.productid
group by p.productname;
```

90 %

Results Messages

	productname	most
1	Headphones	2
2	Laptop	1
3	Mobile	1
4	Tablet	1

-- Correct Answer:

```
select top 3 p.productname, sum(o.quantity) as total_ordered
from products p
join orders o on p.productid = o.productid
group by p.productname
order by total_ordered desc;
```

-- Correct Answer:

```
select top 3 p.productname, sum(o.quantity) as total_ordered
from products p
join orders o on p.productid = o.productid
group by p.productname
order by total_ordered desc;
```

90 %

Results Messages

	productname	total_ordered
1	Headphones	2
2	Tablet	1
3	Mobile	1

-- Q5. Show all orders placed in the last 30 days with customer & product details.

-- wrong answer

-- query was missing the date filter.

-- We must assume the Orders table has an OrderDate column.

```
select c.customername,
p.productname, p.Price
from customers c
left join orders o
on c.customerid=o.customerid
left join products p
on p.productid=o.productid;
```

```
-- Q5. Show all orders placed in the last 30 days with customer & product details.
-- wrong answer
-- query was missing the date filter.
-- We must assume the Orders table has an OrderDate column.
select c.customername, p.productname,p.Price
from customers c
left join orders o
on c.customerid=o.customerid
left join products p
on p.productid=o.productid;
```

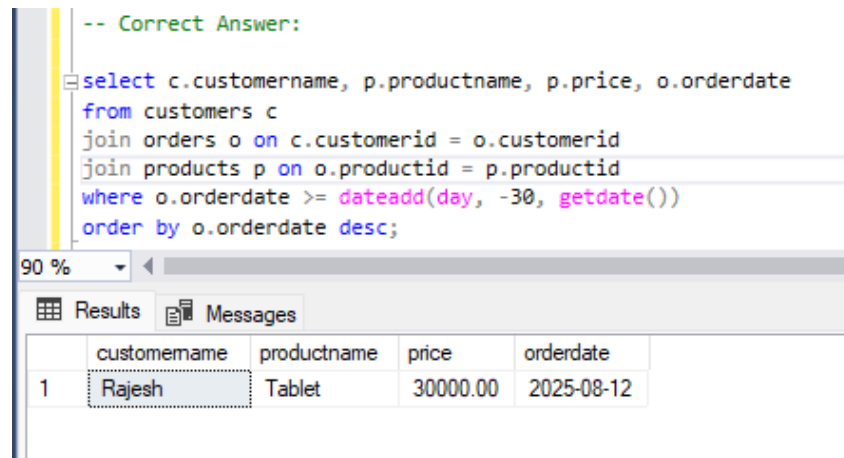
90 %

Results Messages

	customername	productname	Price
1	Suresh	Laptop	55000.00
2	Suresh	Headphones	2000.00
3	Anita	Mobile	20000.00
4	Rajesh	Tablet	30000.00
5	Divya	NULL	NULL

-- Correct Answer:

```
select c.customername, p.productname, p.price, o.orderdate
from customers c
join orders o on c.customerid = o.customerid
join products p on o.productid = p.productid
where o.orderdate >= dateadd(day, -30, getdate())
order by o.orderdate desc;
```



DATEADD in SQL Server (SSMS)

DATEADD(interval, number, date)

- **interval** → what unit to add (day, month, year, etc.)
- **number** → how many units (can be negative for subtracting)
- **date** → starting date

Example:

SELECT GETDATE(); ☞ Suppose today = **2025-09-09 12:30 PM**

SELECT DATEADD(DAY, -30, GETDATE()); ☞ This will subtract 30 days from today = **2025-08-10 12:30 PM**

WHERE o.OrderDate >= DATEADD(DAY, -30, GETDATE());

It means:

☞ Show only orders where **OrderDate is greater than or equal to (today - 30 days)**.

So if today is 2025-09-09, we only want orders from 2025-08-10 till 2025-09-09.

✓ That's why we use DATEADD(DAY, -30, GETDATE()) → it automatically calculates the cutoff date.

-- Dataset 3: Hospitals, Doctors, Patients, Appointments

```
CREATE TABLE Hospitals (  
    HospitalID INT PRIMARY KEY,  
    HospitalName VARCHAR(50),  
    City VARCHAR(50));
```

```
CREATE TABLE Doctors (  
    DoctorID INT PRIMARY KEY,  
    DoctorName VARCHAR(50),  
    Specialty VARCHAR(50),  
    HospitalID INT,  
    FOREIGN KEY (HospitalID) REFERENCES Hospitals(HospitalID));
```

```
CREATE TABLE Patients (  
    PatientID INT PRIMARY KEY,  
    PatientName VARCHAR(50),  
    Age INT);
```

```
CREATE TABLE Appointments (  
    AppointmentID INT PRIMARY KEY,  
    PatientID INT,  
    DoctorID INT,  
    Date DATE,  
    FOREIGN KEY (PatientID) REFERENCES  
Patients(PatientID),  
    FOREIGN KEY (DoctorID) REFERENCES  
Doctors(DoctorID));
```

-- Insert Hospitals

```
INSERT INTO Hospitals VALUES  
(1, 'Apollo', 'Chennai'),  
(2, 'Fortis', 'Bangalore'),  
(3, 'Yashoda', 'Hyderabad');
```

-- Insert Doctors

```
INSERT INTO Doctors VALUES  
(1, 'Dr. Ramesh', 'Cardiology', 1),  
(2, 'Dr. Anita', 'Orthopedics', 1),  
(3, 'Dr. Kiran', 'Neurology', 2),  
(4, 'Dr. Meera', 'Dermatology', 3);
```

-- Insert Patients

```
INSERT INTO Patients VALUES  
(1, 'Akhil', 25),  
(2, 'Sneha', 30),  
(3, 'Vikram', 40),  
(4, 'Pooja', 22);
```

```
select * from hospitals;  
select * from patients;  
select * from doctors;  
select * from appointments;
```

90 %

Results Messages

	HospitalID	HospitalName	City
1	1	Apollo	Chennai
2	2	Fortis	Bangalore
3	3	Yashoda	Hyderabad

	PatientID	PatientName	Age
1	1	Akhil	25
2	2	Sneha	30
3	3	Vikram	40
4	4	Pooja	22

	DoctorID	DoctorName	Specialty	HospitalID
1	1	Dr. Ramesh	Cardiology	1
2	2	Dr. Anita	Orthopedics	1
3	3	Dr. Kiran	Neurology	2
4	4	Dr. Meera	Dermatolo...	3

	AppointmentID	PatientID	DoctorID	Date
1	1	1	1	2025-08-01
2	2	2	2	2025-08-03
3	3	3	3	2025-08-05

```
-- Insert Appointments
INSERT INTO Appointments VALUES
(1, 1, 1, '2025-08-01'),
(2, 2, 2, '2025-08-03'),
(3, 3, 3, '2025-08-05');
```

```
select * from hospitals;
select * from patients;
select * from doctors;
select * from appointments;
```

-- Sample Questions

-- Q1. Show all appointments with patient name, doctor name, and hospital.

```
select p.patientname, d.doctorname, h.hospitalname
from patients p
left join appointments a
on p.patientid=a.patientid
left join doctors d
on d.doctorid=a.doctorid
join hospitals h
on h.hospitalid=d.hospitalid;
```

-- for no appointments

```
select p.patientname,
d.doctorname, h.hospitalname
from patients p
left join appointments a
on p.patientid=a.patientid
left join doctors d
on d.doctorid=a.doctorid
left join hospitals h
on h.hospitalid=d.hospitalid;
```

```
-- Q1. Show all appointments with patient name, doctor name, and hospital.
select p.patientname, d.doctorname, h.hospitalname
from patients p
left join appointments a
on p.patientid=a.patientid
left join doctors d
on d.doctorid=a.doctorid
join hospitals h
on h.hospitalid=d.hospitalid;

-- for no appointments
select p.patientname, d.doctorname, h.hospitalname
from patients p
left join appointments a
on p.patientid=a.patientid
left join doctors d
on d.doctorid=a.doctorid
left join hospitals h
on h.hospitalid=d.hospitalid;
```

82 %

Results Messages

	patientname	doctomame	hospitalname
1	Akhil	Dr. Ramesh	Apollo
2	Sneha	Dr. Anita	Apollo
3	Vikram	Dr. Kiran	Fortis

	patientname	doctomame	hospitalname
1	Akhil	Dr. Ramesh	Apollo
2	Sneha	Dr. Anita	Apollo
3	Vikram	Dr. Kiran	Fortis
4	Pooja	NULL	NULL

-- Q2. Find doctors who don't have any appointments.

```
select d.doctorname, a.appointmentid
from doctors d
left join appointments a
on d.doctorid=a.doctorid
where appointmentid is null;
```

-- Q2. Find doctors who don't have any appointments.

```
select d.doctorname, a.appointmentid
from doctors d
left join appointments a
on d.doctorid=a.doctorid
where appointmentid is null;
```

82 %

Results Messages

	doctorname	appointmentid
1	Dr. Meera	NULL

-- Q3. List all patients with their doctors' specialties.

```
select p.patientname, d.doctorname, d.specialty
from patients p
left join appointments a
on p.patientid=a.patientid
left join doctors d
on d.doctorid=a.doctorid;
```

-- Q3. List all patients with their doctors' specialties.

```
select p.patientname, d.doctorname, d.specialty
from patients p
left join appointments a
on p.patientid=a.patientid
left join doctors d
on d.doctorid=a.doctorid;
```

82 %

Results Messages

	patientname	doctormame	specialty
1	Akhil	Dr. Ramesh	Cardiology
2	Sneha	Dr. Anita	Orthopedics
3	Vikram	Dr. Kiran	Neurology
4	Pooja	NULL	NULL

-- Q4. Find hospitals with no doctors.

```
select h.hospitalname, d.DoctorName
from hospitals h
left join doctors d
on h.hospitalid=d.hospitalid
where d.doctorname is null;
```

no rows in output bcoz all hospitals have doctors

-- Q4. Find hospitals with no doctors.

```
select h.hospitalname, d.DoctorName
from hospitals h
left join doctors d
on h.hospitalid=d.hospitalid
where d.doctorname is null;
```

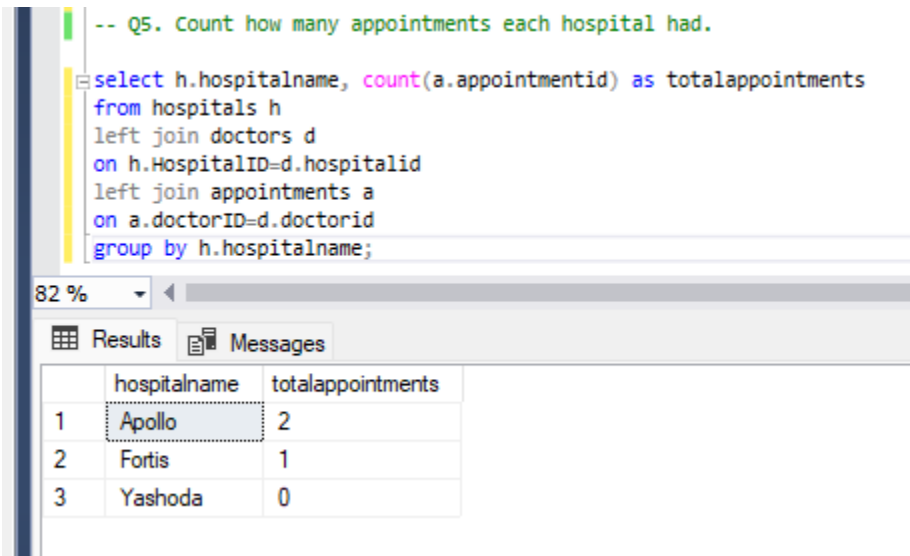
82 %

Results Messages

hospitalname	DoctorName
--------------	------------

-- Q5. Count how many appointments each hospital had.

```
select h.hospitalname, count(a.appointmentid) as totalappointments
from hospitals h
left join doctors d
on h.HospitalID=d.hospitalid
left join appointments a
on a.doctorID=d.doctorid
group by h.hospitalname;
```



The screenshot shows a SQL query editor with a query window and a results window. The query window contains the same SQL query as above. The results window shows the output of the query, which is a table with three columns: hospitalname, totalappointments, and an implicit row number column. The results are as follows:

	hospitalname	totalappointments
1	Apollo	2
2	Fortis	1
3	Yashoda	0